64tass v1.60 r3243 reference manual

This is the manual for 64tass, the multi pass optimizing macro assembler for the 65xx series of processors. Key features:

- Open source portable C with minimal dependencies
- Familiar syntax to Omicron TASS and TASM
- Supports 6502, 65C02, R65C02, W65C02, 65CE02, 65816, DTV, 65EL02, 4510, 45GS02
- Arbitrary-precision integers and bit strings, double precision floating point numbers
- Character and byte strings, array arithmetic
- Handles UTF-8, UTF-16 and 8 bit RAW encoded source files, Unicode character strings
- Supports Unicode identifiers with compatibility normalization and optional case insensitivity
- Built-in “linker” with section support
- Various memory models, binary targets and text output formats (also Hex/S-record)
- Assembly and label listings available for debugging or exporting
- Conditional compilation, macros, structures, unions, scopes

Contrary how the length of this document suggests 64tass can be used with just basic 6502 assembly knowledge in simple ways like any other assembler. If some advanced functionality is needed then this document can serve as a reference.

This is a development version. Features or syntax may change as a result of corrections in non-backwards compatible ways in some rare cases. It's difficult to get everything “right” first time.

Project page: https://sourceforge.net/projects/tass64/

The page hosts the latest and older versions with sources and a bug and a feature request tracker.

1 Table of Contents

2 Usage tips

64tass is a command line assembler, the source can be written in any text editor. As a minimum the source filename must be given on the command line. The “-a” command line option is highly recommended if the source is Unicode or ASCII.

```
64tass -a src.asm
```

There are also some useful parameters which are described later.

For comfortable compiling I use such “Makefile”s (for make):

```
demo.prg: source.asm macros.asm pic.drp music.bin
        64tass -C -a -B -i source.asm -o demo.tmp
        pucrunch -ffast -x 2048 demo.tmp >demo.prg
```

This way “demo.prg” is recreated by compiling “source.asm” whenever “source.asm”, “macros.asm”, “pic.drp” or “music.bin” had changed.

Of course it's not much harder to create something similar for win32 (make.bat), however this will always compile and compress:

```
64tass.exe -C -a -B -i source.asm -o demo.tmp
pucrunch.exe -ffast -x 2048 demo.tmp >demo.prg
```

Here's a slightly more advanced Makefile example with default action as testing in VICE, clean target for removal of temporary files and compressing using an intermediate temporary file:

```
all: demo.prg
        x64 -autostartprgmode 1 -autostart-warp +truedrive +cart $<

demo.prg: demo.tmp
        pucrunch -ffast -x 2048 $< >$@

demo.tmp: source.asm macros.asm pic.drp music.bin
        64tass -C -a -B -i $< -o $@

.INTERMEDIATE: demo.tmp
.PHONY: all clean
clean:
        $(RM) demo.prg demo.tmp
```

It's useful to add a basic header to your source files like the one below, so that the resulting file is directly runnable without additional compression:

```
*       = $0801
        .word (+), 2005  ;pointer, line number
        .null $9e, format("%4d", start);will be sys 4096
+       .word 0          ;basic line end

*       = $1000

start   rts
```

A frequently coming up question is, how to automatically allocate memory, without hacks like `*=*+1`? Sure there's `.byte` and friends for variables with initial values but what about zero page, or RAM outside of program area? The solution is to not use an initial value by using “`?`” or not giving a fill byte value to `.fill`.

```
*       = $02
p1      .addr ?         ;a zero page pointer
temp    .fill 10        ;a 10 byte temporary area
```

Space allocated this way is not saved in the output as there's no data to save at those addresses.

What about some code running on zero page for speed? It needs to be relocated, and the length must be known to copy it there. Here's an example:

```
        ldx #size(zpcode)-1;calculate length
-       lda zpcode,x
        sta wrbyte,x
        dex             ;install to zero page
        bpl -
        jsr wrbyte
        rts
;code continues here but is compiled to run from $02
zpcode  .logical $02
wrbyte  sta $ffff       ;quick byte writer at $02
        inc wrbyte+1
        bne +
        inc wrbyte+2
+       rts
        .endlogical
```

The assembler supports lists and tuples, which does not seems interesting at first as it sound like something which is only useful when heavy scripting is involved. But as normal arithmetic operations also apply on all their elements at once, this could spare quite some typing and repetition.

Let's take a simple example of a low/high byte jump table of return addresses, this usually involves some unnecessary copy/pasting to create a pair of tables with constructs like `>(label-1)`.

```
jumpcmd lda hibytes,x   ; selected routine in X register
        pha
        lda lobytes,x   ; push address to stack
        pha
        rts             ; jump, rts will increase pc by one!
; Build a list of jump addresses minus 1
_       := (cmd_p, cmd_c, cmd_m, cmd_s, cmd_r, cmd_l, cmd_e)-1
lobytes .byte <_        ; low bytes of jump addresses
hibytes .byte >_        ; high bytes
```

There are some other tips below in the descriptions.

3 Expressions and data types

3.1 Integer constants

Integer constants can be entered as decimal digits of arbitrary length. An underscore can be used between digits as a separator for better readability of long numbers. The following operations are accepted:

`x + y`	add `x` to `y`	`2 + 2` is `4`
`x - y`	subtract `y` from `x`	`4 - 1` is `3`
`x * y`	multiply `x` with `y`	`2 * 3` is `6`
`x / y`	integer divide `x` by `y`	`7 / 2` is `3`
`x % y`	integer modulo of `x` divided by `y`	`5 % 2` is `1`
`x ** y`	`x` raised to power of `y`	`2 ** 4` is `16`
`-x`	negated value	`-2` is `-2`
`+x`	unchanged	`+2` is `2`
`~x`	`-x - 1`	`~3` is `-4`
`x | y`	bitwise or	`2 | 6` is `6`
`x ^ y`	bitwise xor	`2 ^ 6` is `4`
`x & y`	bitwise and	`2 & 6` is `2`
`x << y`	logical shift left	`1 << 3` is `8`
`x >> y`	arithmetic shift right	`-8 >> 3` is `-1`

Table 1: Integer operators and functions

Integers are automatically promoted to floats as necessary in expressions. Other types can be converted to integer using the integer type `int`.

Integer division is a floor division (rounding down) so `7 / 4` is `1` and not `1.75`. If ceiling division is required (rounding up) that can be done by negating both the divident and the result. Typically it's done like `0 - -5 / 4` which results in `2`.

```
        .byte 23        ; as unsigned
        .char -23       ; as signed

; using negative integers as immediate values
        ldx #-3         ; works as '#-' is signed immediate
num     = -3
        ldx #+num       ; needs explicit '#+' for signed 8 bits

        lda #((bitmap >> 10) & $0f) | ((screen >> 6) & $f0)
        sta $d018
```

3.2 Bit string constants

Bit string constants can be entered in hexadecimal form with a leading dollar sign or in binary with a leading percent sign. An underscore can be used between digits as a separator for better readability of long numbers. The following operations are accepted:

`~x`	invert bits	`~%101` is `~%101`
`y .. x`	concatenate bits	`$a .. $b` is `$ab`
`y x n`	repeat	`%101 x 3` is `%101101101`
`x[n]`	extract bit(s)	`$a[1]` is `%1`
`x[s]`	slice bits	`$1234[4:8]` is `$3`
`x | y`	bitwise or	`~$2 | $6` is `~$0`
`x ^ y`	bitwise xor	`~$2 ^ $6` is `~$4`
`x & y`	bitwise and	`~$2 & $6` is `$4`
`x << y`	bitwise shift left	`$0f << 4` is `$0f0`
`x >> y`	bitwise shift right	`~$f4 >> 4` is `~$f`

Table 2: Bit string operators and functions

Length of bit string constants are defined in bits and is calculated from the number of bit digits used including leading zeros.

Bit strings are automatically promoted to integer or floating point as necessary in expressions. The higher bits are extended with zeros or ones as needed.

Bit strings support indexing and slicing. This is explained in detail in section "Slicing and indexing".

Other types can be converted to bit string using the bit string type `bits`.

```
        .byte $33        ; 8 bits in hexadecimal
        .byte %00011111 ; 8 bits in binary
        .text $1234      ; $34, $12 (little endian)

        lda $01
        and #~$07        ; 8 bits even after inversion
        ora #$05
        sta $01

        lda $d015
        and #~%00100000 ;clear a bit
        sta $d015
```

3.3 Floating point constants

Floating point constants have a radix point in them and optionally an exponent. A decimal exponent is "e" while a binary one is "p". An underscore can be used between digits as a separator for better readability. The following operations can be used:

`x + y`	add `x` to `y`	`2.2 + 2.2` is `4.4`
`x - y`	subtract `y` from `x`	`4.1 - 1.1` is `3.0`
`x * y`	multiply `x` with `y`	`1.5 * 3` is `4.5`
`x / y`	integer divide `x` by `y`	`7.0 / 2.0` is `3.5`
`x % y`	integer modulo of `x` divided by `y`	`5.0 % 2.0` is `1.0`
`x ** y`	`x` raised to power of `y`	`2.0 ** -1` is `0.5`
`-x`	negated value	`-2.0` is `-2.0`
`+x`	unchanged	`+2.0` is `2.0`
`~x`	almost `-x`	`~2.1` is almost `-2.1`
`x | y`	bitwise or	`2.5 | 6.5` is `6.5`
`x ^ y`	bitwise xor	`2.5 ^ 6.5` is `4.0`
`x & y`	bitwise and	`2.5 & 6.5` is `2.5`
`x << y`	logical shift left	`1.0 << 3.0` is `8.0`
`x >> y`	arithmetic shift right	`-8.0 >> 4` is `-0.5`

Table 3: Floating point operators and functions

As usual comparing floating point numbers for (non) equality is a bad idea due to rounding errors.

The only predefined constant is `pi`.

Floating point numbers are automatically truncated to integer as necessary. Other types can be converted to floating point by using the type `float`.

Fixed point conversion can be done by using the shift operators. For example an 8.16 fixed point number can be calculated as `(3.14 << 16) & $ffffff`. The binary operators operate like if the floating point number would be a fixed point one. This is the reason for the strange definition of inversion.

```
        .byte 3.66e1       ; 36.6, truncated to 36
        .byte $1.8p4       ; 4:4 fixed point number (1.5)
        .sint 12.2p8       ; 8:8 fixed point number (12.2)
```

3.4 Character string constants

Character strings are enclosed in single or double quotes and can hold any Unicode character.

Operations like indexing or slicing are always done on the original representation. The current encoding is only applied when it's used in expressions as numeric constants or in context of text data directives.

Doubling the quotes inside string literals escapes them and results in a single quote.

`y .. x`	concatenate strings	`"a" .. "b"` is `"ab"`
`y in x`	is substring of	`"b" in "abc"` is `true`
`a x n`	repeat	`"ab" x 3` is `"ababab"`

Table 4: Character string operators and functions

a[i]	character from start	"abc"[1] is "b"
a[-i]	character from end	"abc"[-1] is "c"
a[:]	no change	"abc"[:] is "abc"
a[s:]	cut off start	"abc"[1:] is "bc"
a[:-s]	cut off end	"abc"[:-1] is "ab"
a[s]	reverse	"abc"[::-1] is "cba"

Character strings are converted to integers, byte and bit strings as necessary using the current encoding and escape rules. For example when using a sane encoding "z"-"a" is 25.

Other types can be converted to character strings by using the type str or by using the repr and format functions.

Character strings support indexing and slicing. This is explained in detail in section "Slicing and indexing".

```
mystr   = "oeU"         ; character string constant
        .text 'it''s'   ; it's
        .word "ab"+1    ; conversion result is "bb" usually

        .text "text"[:2]     ; "te"
        .text "text"[2:]     ; "xt"
        .text "text"[:-1]    ; "tex"
        .text "reverse"[::-1]; "esrever"
```

3.5 Byte string constants

Byte strings are like character strings, but hold bytes instead of characters.

Quoted character strings prefixing by "b", "l", "n", "p", "s", "x" or "z" characters can be used to create byte strings. The resulting byte string contains what .text, .shiftl, .null, .ptext and .shift would create. Direct hexadecimal entry can be done using the "x" prefix and "z" denotes a z85 encoded byte string. Spaces can be used between pairs of hexadecimal digits as a separator for better readability.

y .. x	concatenate strings	x"12" .. x"34" is x"1234"
y in x	is substring of	x"34" in x"1234" is true
a x n	repeat	x"ab" x 3 is x"ababab"
a[i]	byte from start	x"abcd12"[1] is x"cd"
a[-i]	byte from end	x"abcd"[-1] is x"cd"
a[:]	no change	x"abcd"[:] is x"abcd"
a[s:]	cut off start	x"abcdef"[1:] is x"cdef"
a[:-s]	cut off end	x"abcdef"[:-1] is x"abcd"
a[s]	reverse	x"abcdef"[::-1] is x"efcdab"

Table 5: Byte string operators and functions

Byte strings support indexing and slicing. This is explained in detail in section "Slicing and indexing".

Other types can be converted to byte strings by using the type bytes.

```
        .enc "screen"   ;use screen encoding
mystr   = b"oeU"        ;convert text to bytes, like .text
        .enc "none"     ;normal encoding

        .text mystr     ;text as originally encoded
        .text s"p1"     ;convert to bytes like .shift
        .text l"p2"     ;convert to bytes like .shiftl
        .text n"p3"     ;convert to bytes like .null
        .text p"p4"     ;convert to bytes like .ptext
```

Binary data may be embedded in source code by using hexadecimal byte strings. This is more compact than using .byte followed by a lot of numbers. As expected 1 byte becomes 2 characters.

```
        .text x"fce2"   ;2 bytes: $fc and $e2 (big endian)
```

If readability is not a concern then the more compact z85 encoding may be used which encodes 4 bytes into 5 characters. Data lengths not a multiple of 4 are handled by omitting leading zeros in the last group.

```
        .text z"FiUj*2M$hf";8 bytes: 80 40 20 10 08 04 02 01
```

For data lengths of multiple of 4 bytes any z85 encoder will do. Otherwise the simplest way to encode a binary file into a z85 string is to create a source file which reads it using the line "`label = binary('filename')`". Now if the labels are listed to a file then there will be a z85 encoded definition for this label.

3.6 Lists and tuples

Lists and tuples can hold a collection of values. Lists are defined from values separated by comma between square brackets `[1, 2, 3]`, an empty list is `[]`. Tuples are similar but are enclosed in parentheses instead. An empty tuple is `()`, a single element tuple is `(4,)` to differentiate from normal numeric expression parentheses. When nested they function similar to an array. Both types are immutable.

`y .. x`	concatenate lists	`[1] .. [2]` is `[1, 2]`
`y in x`	is member of list	`2 in [1, 2, 3]` is `true`
`a x n`	repeat	`[1, 2] x 2` is `[1, 2, 1, 2]`
`a[i]`	element from start	`("1", 2)[1]` is `2`
`a[-i]`	element from end	`("1", 2, 3)[-1]` is `3`
`a[:]`	no change	`(1, 2, 3)[:]` is `(1, 2, 3)`
`a[s:]`	cut off start	`(1, 2, 3)[1:]` is `(2, 3)`
`a[:-s]`	cut off end	`(1, 2.0, 3)[:-1]` is `(1, 2.0)`
`a[s]`	reverse	`(1, 2, 3)[::-1]` is `(3, 2, 1)`
`*a`	convert to arguments	`format("%d: %s", *mylist)`
`... op a`	left fold	`... + (1, 2, 3)` is `((1+2)+3)`
`a op ...`	right fold	`(1, 2, 3) - ...` is `(1-(2-3))`

Table 6: List and tuple operators and functions

Arithmetic operations are applied on the all elements recursively, therefore `[1, 2] + 1` is `[2, 3]`, and `abs([1, -1])` is `[1, 1]`.

Arithmetic operations between lists are applied one by one on their elements, so `[1, 2] + [3, 4]` is `[4, 6]`.

When lists form an array and columns/rows are missing the smaller array is stretched to fill in the gaps if possible, so `[[1], [2]] * [3, 4]` is `[[3, 4], [6, 8]]`.

Lists and tuples support indexing and slicing. This is explained in detail in section "Slicing and indexing".

```
mylist  = [1, 2, "whatever"]
mytuple = (cmd_e, cmd_g)

mylist  = ("e", cmd_e, "g", cmd_g, "i", cmd_i)
keys    .text mylist[::2]    ; keys ("e", "g", "i")
call_l  .byte <mylist[1::2]-1; routines (<cmd_e-1, <cmd_g-1, <cmd_i-1)
call_h  .byte >mylist[1::2]-1; routines (>cmd_e-1, >cmd_g-1, >cmd_i-1)
```

Although lists elements of variables can't be changed using indexing (at the moment) the same effect can be achieved by combining slicing and concatenation:

```
lst     := lst[:2] .. [4] .. lst[3:]; same as lst[2] := 4 would be
```

Folding is done on pair of elements either forward (left) or reverse (right). The list must contain at least one element. Here are some folding examples:

```
minimum = size([part1, part2, part3]) <? ...
```

```
maximum = size([part1, part2, part3]) >? ...
sum     = size([part1, part2, part3]) + ...
xorall  = list_of_numbers ^ ...
join    = list_of_strings .. ...
allbits = sprites.(left, middle, right).bits | ...
all     = [true, true, true, true] && ...
any     = [false, false, false, true] || ...
```

The `range(start, end, step)` built-in function can be used to create lists of integers in a range with a given step value. At least the end must be given, the start defaults to 0 and the step to 1. Sounds not very useful, so here are a few examples:

```
;Bitmask table, 8 bits from left to right
        .byte %10000000 >> range(8)
;Classic 256 byte single period sinus table with values of 0-255.
        .byte 128 + 127.5 * sin(range(256) * pi / 128)
;Screen row address tables
_       := $400 + range(0, 1000, 40)
scrlo   .byte <_
scrhi   .byte >_
```

3.7 Dictionaries

Dictionaries hold key and value pairs normally but can be used as sets too if simple values are used. In the latter case the values are the keys for themselves.

A dictionary is defined with coma separated values between curly brackets. An empty one is `{}`. Key and value pairs are separated with colon, like `{ <key>:<value> }`. A default value for missing items is can be defined by leaving out the key before the colon, like `{ :<default> }`. Simple value don't use a colon `{ <value> }`.

Looking up a non-existing key is an error unless a default value is given. Dictionaries are immutable. There are limitations what may be used as a key but the value can be anything. As the keys are used for lookups these must be unique.

`y .. x`	combine dictionaries	`{1:2, 3:4} .. {2:3, 3:1}` is `{1:2, 2:3, 3:1}`
`x[i]`	value lookup	`{"1":2}["1"]` is `2`
`x.i`	symbol lookup	`{.ONE:1, .TWO:2}.ONE` is `1`
`y in x`	is a key	`1 in {1:2}` is `true`

Table 7: Dictionary operators and functions

```
; Simple lookup
        .text {1:"one", 2:"two"}[2]; "two"
; 16 element "fader" table 1->15->12->11->0
        .byte {1:15, 15:12, 12:11, :0}[range(16)]
; Variables can be used to build dictionaries incrementally.
md      := {1:2}
md      ..= {3:4}
```

The keys can be symbols as well, this allows simple definition of data structures or enumerations.

```
; Symbol accessible values. May be useful as a function return value too.
coords  = {.x: 24, .y: 50}
        ldx #coords.x
        ldy #coords.y
; Simple enumeration where red = 0, green = 1, blue = 2
colors  = dict(.(red, green, blue), range(3))
        lda #color.green
; Enumerate register bits as %1, %10, %100, ...
irqbits = dict(.(ta, tb, tod, serial, flag), %1 << range(5))
        and #irqbits.flag
```

3.8 Code

Code holds the result of compilation in binary and other enclosed objects. In an arithmetic operation it's used as the numeric address of the memory where it starts. The compiled content remains static even if later parts of the source overwrite the same memory area.

Indexing and slicing of code to access the compiled content might be implemented differently in future releases. Use this feature at your own risk for now, you might need to update your code later.

`a.b`	b member of a	`label.locallabel`
`.b in a`	if a has symbol b	`.locallabel in label`
`a[i]`	element from start	`label[1]`
`a[-i]`	element from end	`label[-1]`
`a[:]`	copy as tuple	`label[:]`
`a[s:]`	cut off start, as tuple	`label[1:]`
`a[:-s]`	cut off end, as tuple	`label[:-1]`
`a[s]`	reverse, as tuple	`label[::-1]`

Table 8: Label operators and functions

```
mydata  .word 1, 4, 3
mycode  .block
local   lda #0
        .endblock

        ldx #size(mydata) ;6 bytes (3*2)
        ldx #len(mydata)  ;3 elements
        ldx #mycode[0]    ;lda instruction, $a9
        ldx #mydata[1]    ;2nd element, 4
        jmp mycode.local  ;address of local label
```

3.9 Addressing modes

Addressing modes are used for determining addressing modes of instructions.

For indexing there must be no white space between the comma and the register letter, otherwise the indexing operator is not recognized. On the other hand put a space between the comma and a single letter symbol in a list to avoid it being recognized as an operator.

`#`	immediate
`#+`	signed immediate
`#-`	signed immediate
`( )`	indirect
`[ ]`	long indirect
`,b`	data bank indexed
`,d`	direct page indexed
`,k`	program bank indexed
`,r`	data stack pointer indexed
`,s`	stack pointer indexed
`,x`	x register indexed
`,y`	y register indexed
`,z`	z register indexed

Table 9: Addressing mode operators

Parentheses are used for indirection and square brackets for long indirection. These operations are only available after instructions and functions to not interfere with their normal use in expressions.

Several addressing mode operators can be combined together. **Currently the complexity is limited to 4 operators. This is enough to describe all addressing modes of the supported CPUs.**

`#`	immediate	`lda #$12`
`#+`	signed immediate	`lda #+127`
`#-`	signed immediate	`lda #-128`

Table 10: Valid addressing mode operator combinations

`#addr,#addr`	move	`mvp #5,#6`
`addr`	direct or relative	`lda $12 lda $1234 bne $1234`
`bit,addr`	direct page bit	`rmb 5,$12`
`bit,addr,addr`	direct page bit relative jump	`bbs 5,$12,$1234`
`(addr)`	indirect	`lda ($12) jmp ($1234)`
`(addr),y`	indirect y indexed	`lda ($12),y`
`(addr),z`	indirect z indexed	`lda ($12),z`
`(addr,x)`	x indexed indirect	`lda ($12,x) jmp ($1234,x)`
`[addr]`	long indirect	`lda [$12] jmp [$1234]`
`[addr],y`	long indirect y indexed	`lda [$12],y`
`#addr,b`	data bank indexed	`lda #0,b`
`#addr,b,x`	data bank x indexed	`lda #0,b,x`
`#addr,b,y`	data bank y indexed	`lda #0,b,y`
`#addr,d`	direct page indexed	`lda #0,d`
`#addr,d,x`	direct page x indexed	`lda #0,d,x`
`#addr,d,y`	direct page y indexed	`ldx #0,d,y`
`(#addr,d)`	direct page indirect	`lda (#$12,d)`
`(#addr,d,x)`	direct page x indexed indirect	`lda (#$12,d,x)`
`(#addr,d),y`	direct page indirect y indexed	`lda (#$12,d),y`
`(#addr,d),z`	direct page indirect z indexed	`lda (#$12,d),z`
`[#addr,d]`	direct page long indirect	`lda [#$12,d]`
`[#addr,d],y`	direct page long indirect y indexed	`lda [#$12,d],y`
`#addr,k`	program bank indexed	`jsr #0,k`
`(#addr,k,x)`	program bank x indexed indirect	`jmp (#$1234,k,x)`
`#addr,r`	data stack indexed	`lda #1,r`
`(#addr,r),y`	data stack indexed indirect y indexed	`lda (#$12,r),y`
`#addr,s`	stack indexed	`lda #1,s`
`(#addr,s),y`	stack indexed indirect y indexed	`lda (#$12,s),y`
`addr,x`	x indexed	`lda $12,x`
`addr,y`	y indexed	`lda $12,y`

Direct page, data bank, program bank indexed and long addressing modes of instructions are intelligently chosen based on the instruction type, the address ranges set up by `.dpage`, `.databank` and the current program counter address. Therefore the “`,d`”, “`,b`” and “`,k`” indexing is only used in very special cases.

The immediate direct page indexed “`#0,d`” addressing mode is usable for direct page access. The 8 bit constant is a direct offset from the start of actual direct page. Alternatively it may be written as “`0,d`”.

The immediate data bank indexed “`#0,b`” addressing mode is usable for data bank access. The 16 bit constant is a direct offset from the start of actual data bank. Alternatively it may be written as “`0,b`”.

The immediate program bank indexed “`#0,k`” addressing mode is usable for program bank jumps, branches and calls. The 16 bit constant is a direct offset from the start of actual program bank. Alternatively it may be written as “`0,k`”.

The immediate stack indexed “`#0,s`” and data stack indexed “`#0,r`” accept 8 bit constants as an offset from the start of (data) stack. These are sometimes written without the immediate notation, but this makes it more clear what's going on. For the same reason the move instructions are written with an immediate addressing mode “`#0,#0`” as well.

The immediate (`#`) addressing mode expects unsigned values of byte or word size. Therefore it only accepts constants of 1 byte or in range 0–255 or 2 bytes or in range 0–65535.

The signed immediate (`#+` and `#-`) addressing mode is to allow signed numbers to be used as immediate constants. It accepts a single byte or an integer in range −128–127, or two bytes or an integer of −32768–32767.

The use of signed immediate (like `#-3`) is seamless, but it needs to be explicitly written out for variables or expressions (`#+variable`). In case the unsigned variant is needed but the expression starts with a negation then it needs to be put into parentheses (`#(-variable)`) or else it'll change the address mode to signed.

Normally addressing mode operators are used in expressions right after instructions. They can also be used for defining stack variable symbols when using a 65816, or to force a specific addressing mode.

```
param   = #1,s              ;define a stack variable
const   = #1                ;immediate constant
        lda #0,b            ;always "absolute" lda $0000
        lda param           ;results in lda #$01,s
        lda param+1         ;results in lda #$02,s
        lda (param),y       ;results in lda (#$01,s),y
        ldx const           ;results in ldx #$01
        lda #-2             ;negative constant, $fe
```

3.10 Uninitialized memory

There's a special value for uninitialized memory, it's represented by a question mark. Whenever it's used to generate data it creates a "hole" where the previous content of memory is visible.

Uninitialized memory holes without previous content are not saved unless it's really necessary for the output format, in that case it's replaced with zeros.

It's not just data generation statements (e.g. `.byte`) that can create uninitialized memory, but `.fill`, `.align` or address manipulation as well.

```
*       = $200          ;bytes as necessary
        .word ?         ;2 bytes
        .fill 10        ;10 bytes
        .align 64       ;bytes as necessary
```

3.11 Booleans

There are two predefined boolean constant variables, `true` and `false`.

Booleans are created by comparison operators (`<`, `<=`, `!=`, `==`, `>=`, `>`), logical operators (`&&`, `||`, `^^`, `!`), the membership operator (`in`) and the `all` and `any` functions.

Normally in numeric expressions `true` is `1` and `false` is `0`, unless the "`-Wstrict-bool`" command line option was used.

Other types can be converted to boolean by using the type `bool`.

`bits`	At least one non-zero bit
`bool`	When true
`bytes`	At least one non-zero byte
`code`	Address is non-zero
`float`	Not `0.0`
`int`	Not zero
`str`	At least one non-zero byte after translation

Table 11: Boolean values of various types

3.12 Types

The various types mentioned earlier have predefined names. These can used for conversions or type checks.

`address`	Address type
`bits`	Bit string type
`bool`	Boolean type
`bytes`	Byte string type
`code`	Code type
`dict`	Dictionary type
`float`	Floating point type
`gap`	Uninitialized memory type

Table 12: Built-in type names

`int`	Integer type
`list`	List type
`str`	Character string type
`symbol`	Symbol type
`tuple`	Tuple type
`type`	Type type

Bit and byte string conversions can take a second parameter to specify an exact size. Values which can fit in shorter space will be padded but longer ones give an error.

bits(`<expression>[, <bit count>]`**)**
: Convert to the specific number of bits. If the number of bits is negative then it's a signed.

bytes(`<expression>[, <byte count>]`**)**
: Convert to the specific number of bytes. If the number of bits is negative then it's a signed.

Dictionaries can be built from a single iterable of key and value pairs, or from two iterables where the keys come from the first and the values from the second parameter.

dict(`<iterable>[, <values iterable>]`**)**
: Build dictionary from iterables

```
        .cerror type(var) != str, "Not a string!"
        .text str(year)   ; convert to string
```

3.13 Symbols

Symbols are used to reference objects. Regularly named, anonymous and local symbols are supported. These can be constant or re-definable.

Scopes are where symbols are stored and looked up. The global scope is always defined and it can contain any number of nested scopes.

Symbols must be uniquely named in a scope, therefore in big programs it's hard to come up with useful and easy to type names. That's why local and anonymous symbols exists. And grouping certain related symbols into a scope makes sense sometimes too.

Scopes are usually created by `.proc` and `.block` directives, but there are a few other ways. Symbols in a scope can be accessed by using the dot operator, which is applied between the name of the scope and the symbol (e.g. `myconsts.math.pi`).

3.13.1 Regular symbols

Regular symbol names are starting with a letter and containing letters, numbers and underscores. Unicode letters are allowed if the “`-a`” command line option was used. There's no restriction on the length of symbol names.

Care must be taken to not use duplicate names in the same scope when the symbol is used as a constant as there can be only one definition for them.

Duplicate names in parent scopes are not a problem and this gives the ability to override names defined in lower scopes. However this can just as well lead to mistakes if a lower scoped symbol with the same name was meant so there's a “`-Wshadow`” command line option to warn if such ambiguity exists.

Case sensitivity can be enabled with the “`-C`” command line option, otherwise all symbols are matched case insensitive.

For case insensitive matching it's possible to check for consistent symbol name use with the “`-Wcase-symbol`” command line option.

A regular symbol is looked up first in the current scope, then in lower scopes until the global scope is reached.

```
f       .block
g        .block
n        nop            ;jump here
         .endblock
```

```
        .endblock

        jsr f.g.n       ;reference from a scope
f.x     = 3             ;create x in scope f with value 3
```

3.13.2 Local symbols

Local symbols have their own scope between two regularly named code symbols and are assigned to the code symbol above them.

Therefore they're easy to reuse without explicit scope declaration directives.

Not all regularly named symbols can be scope boundaries just plain code symbol ones without anything or an opcode after them (no macros!). Symbols defined as procedures, blocks, macros, functions, structures and unions are ignored. Also symbols defined by `.var`, `:=` or `=` don't apply, and there are a few more exceptions, so stick to using plain code labels.

The name must start with an underscore (_), otherwise the same character restrictions apply as for regular symbols. There's no restriction on the length of the name.

Care must be taken to not use the duplicate names in the same scope when the symbol is used as a constant.

A local symbol is only looked up in it's own scope and nowhere else.

```
incr    inc ac
        bne _skip
        inc ac+1
_skip   rts

decr    lda ac
        bne _skip
        dec ac+1
_skip   dec ac          ;symbol reused here
        jmp incr._skip  ;this works too, but is not advised
```

3.13.3 Anonymous symbols

Anonymous symbols don't have a unique name and are always called as a single plus or minus sign. They are also called as forward (+) and backward (-) references.

When referencing them “-” means the first backward, “--” means the second backwards and so on. It's the same for forward, but with “+”. In expressions it may be necessary to put them into brackets.

```
        ldy #4
-       ldx #0
-       txa
        cmp #3
        bcc +
        adc #44
+       sta $400,x
        inx
        bne -
        dey
        bne --
```

Excessive nesting or long distance references create poorly readable code. It's also very easy to copy-paste a few lines of code with these references into a code fragment already containing similar references. The result is usually a long debugging session to find out what went wrong.

These references are also useful in segments, but this can create a nice trap when segments are copied into the code with their internal references.

```
        bne +
        #somemakro      ;let's hope that this segment does
+       nop             ;not contain forward references...
```

Anonymous symbols are looked up first in the current scope, then in lower scopes until the global scope is reached.

Anonymous labels within conditionally assembled code are counted even if the code itself is not compiled and the label won't get defined. This ensures that anonymous labels are always at the same "distance" independent of the conditions in between.

3.13.4 Constant and re-definable symbols

Constant symbols can be created with the equal sign. These are not re-definable. Forward referencing of them is allowed as they retain the objects over compilation passes.

Symbols in front of code or certain assembler directives are created as constant symbols too. They are bound to the object following them.

Re-definable symbols can be created by the `.var` directive or `:=` construct. These are also called as variables. They don't carry their content over from the previous pass therefore it's not possible to use them before their definition.

If the variable already exists in the current scope it'll get updated. If an existing variable needs to be updated in a parent scope then the `::=` variable reassign operator is able to do that.

Variables can be conditionally defined using the `:?=` construct. If the variable was defined already then the original value is retained otherwise a new one is created with this value.

```
WIDTH   = 40            ;a constant
        lda #WIDTH      ;lda #$28
variabl .var 1          ;a variable
var2    := 1            ;another variable
variabl .var variabl + 1;update it verbosely
var2    += 1            ;compound assignment (add one)
var3    :?= 5           ;assign 5 if undefined
```

3.13.5 The star label

The “*” symbol denotes the current program counter value. When accessed it's value is the program counter at the beginning of the line. Assigning to it changes the program counter and the compiling offset.

3.14 Built-in functions

Built-in functions are pre-assigned to the symbols listed below. If you reuse these symbols in a scope for other purposes then they become inaccessible, or can perform a different function.

Built-in functions can be assigned to symbols (e.g. `sinus = sin`), and the new name can be used as the original function. They can even be passed as parameters to functions.

3.14.1 Mathematical functions

floor(<expression>**)**
: Round down. E.g. `floor(-4.8)` is `-5.0`

round(<expression>**)**
: Round to nearest away from zero. E.g. `round(4.8)` is `5.0`

ceil(<expression>**)**
: Round up. E.g. `ceil(1.1)` is `2.0`

trunc(<expression>**)**
: Round down towards zero. E.g. `trunc(-1.9)` is `-1`

frac(<expression>**)**
: Fractional part. E.g. `frac(1.1)` is `0.1`

sqrt(<expression>**)**
: Square root. E.g. `sqrt(16.0)` is `4.0`

cbrt(<expression>**)**
: Cube root. E.g. `cbrt(27.0)` is `3.0`

log10(<expression>**)**
: Common logarithm. E.g. `log10(100.0)` is `2.0`

log(<expression>[, <expression base>]**)**
: Logarithm, natural by default. E.g. `log(1)` is `0.0`

exp(<expression>**)**
: Exponential. E.g. `exp(0)` is `1.0`

pow(<expression a>, <expression b>**)**
: A raised to power of B. E.g. `pow(2.0, 3.0)` is `8.0`

sin(<expression>**)**
: Sine. E.g. `sin(0.0)` is `0.0`

asin(<expression>**)**
: Arc sine. E.g. `asin(0.0)` is `0.0`

sinh(<expression>**)**
: Hyperbolic sine. E.g. `sinh(0.0)` is `0.0`

cos(<expression>**)**
: Cosine. E.g. `cos(0.0)` is `1.0`

acos(<expression>**)**
: Arc cosine. E.g. `acos(1.0)` is `0.0`

cosh(<expression>**)**
: Hyperbolic cosine. E.g. `cosh(0.0)` is `1.0`

tan(<expression>**)**
: Tangent. E.g. `tan(0.0)` is `0.0`

atan(<expression>**)**
: Arc tangent. E.g. `atan(0.0)` is `0.0`

tanh(<expression>**)**
: Hyperbolic tangent. E.g. `tanh(0.0)` is `0.0`

rad(<expression>**)**
: Degrees to radian. E.g. `rad(0.0)` is `0.0`

deg(<expression>**)**
: Radian to degrees. E.g. `deg(0.0)` is `0.0`

hypot([<expression>, …]**)**
: Euclidean distance, any dimensions. E.g. `hypot(4.0, 3.0)` is `5.0`

atan2(<expression y>, <expression x>**)**
: Polar angle in −pi to +pi range. E.g. `atan2(0.0, 3.0)` is `0.0`

abs(<expression>**)**
: Absolute value. E.g. `abs(-1)` is `1`

sign(<expression>**)**
: Returns the sign of value as −1, 0 or 1 for negative, zero and positive. E.g. `sign(-5)` is `-1`

3.14.2 Byte string functions

These functions return byte strings of various lengths for signed numbers, unsigned numbers and addresses.

The naming of functions is not a coincidence and they return the bytes what the data directives with the same names normally emit.

byte(<expression>**)**
char(<expression>**)**
: Return a single byte string from a 8 bit unsigned (0–255) or signed number (−128–127). E.g. `byte(0)` is `x"00"` and `char(-1)` is `x"ff"`

word(<expression>**)**
sint(<expression>**)**
: Return a little endian byte string of 2 bytes from a 16 bit unsigned (0–65535) or signed number (−32768–32767). E.g. `word(1024)` is `x"0004"` and `sint(-1)` is `x"ffff"`

long(<expression>**)**
lint(<expression>**)**

Return a little endian byte string of 3 bytes from a 24 bit unsigned (0–16777216) or signed number (−8388608–8388607). E.g. `long(123456)` is `x"40E201"` and `lint(-1)` is `x"ffffff"`

dword(<expression>**)**
dint(<expression>**)**

Return a little endian byte string of 4 bytes from a 32 bit unsigned (0–4294967296) or signed number (−2147483648–2147483647). E.g. `dword(123456789)` is `x"15CD5B07"` and `dint(-1)` is `x"ffffffff"`

addr(<expression>**)**

Return a little endian byte string of 2 bytes from an address in the current program bank. E.g. `addr(start)` is `x"0d08"`

rta(<expression>**)**

Return a little endian byte string of 2 bytes from a return address in the current program bank. E.g. `rta(4096)` is `x"ff0f"`

3.14.3 Other functions

all(<expression>**)**

Return truth for various definitions of “all”.

all bits set or no bits at all	`all($f)` is `true`
all characters non-zero or empty string	`all("c")` is `true`
all bytes non-zero or no bytes	`all(x"ac24")` is `true`
all elements true or empty list	`all([true, true, false])` is `false`

Table 13: All function

Only booleans in a list are accepted with the “`-Wstrict-bool`” command line option.

any(<expression>**)**

Return truth for various definitions of “any”.

at least one bit set	`any(~$f)` is `false`
at least one non-zero character	`any("c")` is `true`
at least one non-zero byte	`any(x"ac24")` is `true`
at least one true element	`any([true, true, false])` is `true`

Table 14: Any function

Only booleans in a list are accepted with the “`-Wstrict-bool`” command line option.

binary(<string expression>[, <offset>[, <length>]]**)**

Returns the binary file content as bytes.

This function reads the content of a binary file as a byte string. It also accepts optional offset and length parameters.

Read everything	`binary(name)`
Skip starting bytes	`binary(name, offset)`
Some bytes from offset	`binary(name, offset, length)`

Table 15: Binary function invocation types

```
sid     = binary("music.sid"); read in the SID file as bytes
offs    := sid[[$7, $6]]      ; data offset (big endian)
load    := sid[[$9, $8]]      ; load address (big endian)
init    = sid[[$b, $a]]       ; init address (big endian)
play    = sid[[$d, $c]]       ; play address (big endian)

; if load address is zero then it's the first 2 bytes of data
        .if load == 0
load    := sid[offs:offs+2]   ; load address (little endian)
offs    += 2                  ; skip load address bytes
        .endif
```

```
*       = load              ; set pc to load address
        .text sid[offs:]    ; dump music data
```

format(<string expression>[, <expression>, …])

Create string from values according to a format string.

The format function converts a list of values into a character string. The converted values are inserted in place of the % sign. Optional conversion flags and minimum field length may follow, before the conversion type character. These flags can be used:

#	alternate form (-$a, ~$a, -%10, ~%10, -10.)
*	width/precision from list
.	precision
0	pad with zeros
-	left adjusted (default right)
	blank when positive or minus sign
+	sign even if positive
~	binary and hexadecimal as bits

Table 16: Formatting flags

The following conversion types are implemented:

b	binary
c	Unicode character
d	decimal
e E	exponential float (uppercase)
f F	floating point (uppercase)
g G	exponential/floating point
s	string
r	representation
x X	hexadecimal (uppercase)
%	percent sign

Table 17: Formatting conversion types

```
        .text format("%#04x bytes left", 1000); $03e8 bytes left
```

len(<expression>)

Returns the number of elements.

bit string	length in bits	len($034) is 12
character string	number of characters	len("abc") is 3
byte string	number of bytes	len(x"abcd23") is 3
tuple, list	number of elements	len([1, 2, 3]) is 3
dictionary	number of elements	len({1:2, 3:4]) is 2
code	number of elements	len(label)

Table 18: Length of various types

random([<expression>, …])

Returns a pseudo random number.

The sequence does not change across compilations and is the same every time. Different sequences can be generated by seeding with .seed.

floating point number 0.0 <= x < 1.0	random()
integer in range of 0 <= x < e	random(e)
integer in range of s <= x < e	random(s, a)
integer in range of s <= x < e, step t	random(s, a, t)

Table 19: Random function invocation types

```
        .seed 1234      ; default is boring, seed the generator
        .byte random(256); a pseudo random byte (0-255)
        .byte random([16] x 8); 8 pseudo random bytes (0-15)
```

range(<expression>[, <expression>, …]**)**

Returns a list of integers in a range, with optional stepping.

integers from 0 to e-1	range(e)
integers from s to e-1	range(s, a)
integers from s to e (not including e), step t	range(s, a, t)

Table 20: Range function invocation types

```
        .byte range(16) ; 0, 1, ..., 14, 15
        .char range(-5, 6); -5, -4, ..., 4, 5
mylist  = range(10, 0, -2); [10, 8, 6, 4, 2]
```

repr(<expression>**)**

Returns a string representation of value.

```
        .warn repr(var) ; pretty print value, for debugging
```

size(<expression>**)**

Returns the size of code, structure or union in bytes.

```
var     .word 0, 0, 0
        ldx #size(var)  ; 6 bytes
var2    = var + 2       ; start 2 bytes later
        ldx #size(var2) ; what remains is 4 bytes
```

sort(<list expression>**)**

Returns a sorted list or tuple.

If the original list contains further lists then these must be all of the same length. In this case the order of lists is determined by comparing their elements from the start until a difference is found. The sort is stable.

```
; sort IRQ routines by their raster lines
sorted  = sort([(60, irq1), (50, irq2)])
lines   .byte sorted[:, 0] ; 50, 60
irqs    .addr sorted[:, 1] ; irq2, irq1
```

3.15 Expressions

3.15.1 Operators

The following operators are available. Not all are defined for all types of arguments and their meaning might slightly vary depending on the type.

-	negative	+	positive
!	not	~	invert
*	convert to arguments	^	*decimal string*

Table 21: Unary operators

The "^" decimal string operator will be changed to mean the bank byte soon. Please update your sources to use `format("%d", xxx)` instead! This is done to be in line with it's use in most other assemblers.

+	add	-	subtract
*	multiply	/	divide
%	modulo	**	raise to power
|	binary or	^	binary xor
&	binary and	<<	shift left
>>	shift right	.	member
..	concat	x	repeat
in	contains	!in	excludes

Table 22: Binary operators

Spacing must be used for the “`x`” and “`in`” operators or else they won't be recognized as such. For example the expression “`[1,2]x2`” should be written as “`[1,2]x 2`” instead.

Parenthesis (`( )`) can be used to override operator precedence. Don't forget that they also denote indirect addressing mode for certain opcodes.

```
        lda #(4+2)*3
```

3.15.2 Comparison operators

Traditional comparison operators give false or true depending on the result.

The compare operator (`<=>`) gives −1 for less, 0 for equal and 1 for more.

`<=>`	compare		
`==`	equals	`!=`	not equal
`<`	less than	`>=`	more than or equals
`>`	more than	`<=`	less than or equals
`===`	identical	`!==`	not identical

Table 23: Comparison operators

3.15.3 Bit string extraction operators

These unary operators extract 8 or 16 bits. Usually they are used to get parts of a memory address.

`<`	lower byte	`>`	higher byte
`<>`	lower word	``>` ``	higher word
`><`	lower byte swapped word	`` ` ``	bank byte

Table 24: Bit string extraction operators

```
        lda #<label     ; low byte of address
        ldy #>label     ; high byte of address
        jsr $ab1e

        ldx #<>source   ; word extraction
        ldy #<>dest
        lda #size(source)-1
        mvn #`source, #`dest; bank extraction
```

Please note that these prefix operators are not strongly binding like negation or inversion. Instead they apply to the whole expression to the right. This may be unexpected but is required for compatibility with old sources which expect this behaviour.

```
        lda #<label+10  ;This is <(label+10) and not (<label)+10

;The check below is wrong and should be written as (>start) != (>end)
        .cerror >start != >end;Effectively this is >(start != (>end))
```

3.15.4 Conditional operators

Boolean conditional operators give false or true or one of the operands as the result.

`x || y`	if `x` is true then `x` otherwise `y`
`x ^^ y`	if both false or true then `false` otherwise `x || y`
`x && y`	if `x` is true then `y` otherwise `x`
`!x`	if `x` is true then `false` otherwise `true`
`c ? x : y`	if `c` is true then `x` otherwise `y`
`c ?? x : y`	if `c` is true then `x` otherwise `y` (broadcasting)
`x <? y`	if `x` is smaller then `x` otherwise `y`
`x >? y`	if `x` is greater then `x` otherwise `y`

Table 25: Logical and conditional operators

```
;Silly example for 1=>"simple", 2=>"advanced", else "normal"
        .text MODE == 1 && "simple" || MODE == 2 && "advanced" || "normal"
        .text MODE == 1 ? "simple" : MODE == 2 ? "advanced" : "normal"
;Limit result to 0 .. 8
light   .byte 0 >? range(-16, 101)/6 <? 8
```

Please note that these are not short circuiting operations and both sides are calculated even if thrown away later.

With the "`-Wstrict-bool`" command line option booleans are required as arguments and only the "`?`" operator may return something else.

3.15.5 Address length forcing

Special addressing length forcing operators in front of an expression can be used to make sure the expected addressing mode is used. Only applicable when used directly at the mnemonic.

`@b`	to force 8 bit address
`@w`	to force 16 bit address
`@l`	to force 24 bit address (65816)

Table 26: Address size forcing

```
        lda @w $0000    ; force the use of 2 byte absolute addressing
        bne @b label    ; prevent upgrade to beq+jmp with long branches in use
        lda @w #$00     ; use 2 bytes independent of accumulator size
```

3.15.6 Compound assignment

These assignment operators are short hands for updating variables. Constants can't be changed of course.

The variables on the left must be defined beforehand by "`:=`" or "`.var`".

Compound assignment operators can modify variables defined in parent scopes as well.

`+=`	add	`-=`	subtract
`*=`	multiply	`/=`	divide
`%=`	modulo	`**=`	raise to power
`|=`	binary or	`^=`	binary xor
`&=`	binary and	`||=`	logical or
`&&=`	logical and	`<<=`	shift left
`>>=`	shift right	`..=`	concat
`<?=`	smaller	`>?=`	greater
`x=`	repeat	`.=`	member

Table 27: Compound assignments

```
v       += 1            ; same as 'v ::= v + 1'
```

3.15.7 Slicing and indexing

Lists, character strings, byte strings and bit strings support various slicing and indexing possibilities through the `[]` operator.

Indexing elements with positive integers is zero based. Negative indexes are transformed to positive by adding the number of elements to them, therefore −1 is the last element. Indexing with list of integers is possible as well so `[1, 2, 3][(-1, 0, 1)]` is `[3, 1, 2]`.

Slicing is an operation when parts of sequence is extracted from a start position to an end position with a step value. These parameters are separated with colons enclosed in square brackets and are all optional. Their default values are `[start:maximum:step=1]`. Negative start and end characters are converted to positive internally by adding the length of string to them. Negative step operates in reverse direction, non-single steps will jump over elements.

This is quite powerful and therefore a few examples will be given here:

Positive indexing `a[x]`
: It'll simply extracts a numbered element. It is zero based, therefore `"abcd"[1]` results in `"b"`.

Negative indexing `a[-x]`
: This extracts an element counted from the end, −1 is the last one. So `"abcd"[-2]` results in `"c"`.

Cut off end `a[:to]`
: Extracts a continuous range stopping before “to”. So `[10,20,30,40][:-1]` results in `[10,20,30]`.

Cut off start `a[from:]`
: Extracts a continuous range starting from “from”. So `[10,20,30,40][-2:]` results in `[30,40]`.

Slicing `a[from:to]`
: Extracts a continuous range starting from element “from” and stopping before “to”. The two end positions can be positive or negative indexes. So `[10,20,30,40][1:-1]` results in `[20,30]`.

Everything `a[:]`
: Giving no start or end will cover everything and therefore results in a complete copy.

Reverse `a[::-1]`
: This gives everything in reverse, so `"abcd"[::-1]` is `"dcba"`.

Stepping through `a[from:to:step]`
: Extracts every “step”th element starting from “from” and stopping before “to”. So `"abcdef"[1:4:2]` results in `"bd"`. The “from” and “to” can be omitted in case it starts from the beginning or end at the end. If the “step” is negative then it's done in reverse.

Extract multiple elements `a[list]`
: Extract elements based on a list. So `"abcd"[[1,3]]` will be `"bd"`.

The fun start with nested lists and tuples, as these can be used to create a matrix. The examples will be given for a two dimensional matrix for easier understanding, but this also works in higher dimensions.

Extract row `a[x]`
: Given a `[(1,2),(3,4)]` matrix `[0]` will give the first row which is `(1,2)`

Extract row range `a[from:to]`
: Given a `[(1,2),(3,4),(5,6),(7,8)]` matrix `[1:3]` will give `[(3,4),(5,6)]`

Extract column `a[x]`
: Given a `[(1,2),(3,4)]` matrix `[:,0]` will give the first column of all rows which is `[1,3]`

Extract column range `a[:,from:to]`
: Given a `[(1,2,3,4),(5,6,7,8)]` matrix `[:,1:3]` will give `[(2,3),(6,7)]`

And it works for list of indexes, negative indexes, stepped ranges, reversing, etc. on all axes in too many ways to show all possibilities.

Basically it's just the indexing and slicing applied on nested constructs, where each nesting level is separated by a comma.

4 Compiler directives

4.1 Controlling the compile offset and program counter

Two counters are used while assembling.

The compile offset is where the data and code ends up in memory (or in image file).

The program counter is what labels get set to and what the special star label refers to.

Normally both are the same (code is compiled to the location it runs from) but it does not need to be.

`*= <expression>`
: The compile offset is adjusted so that the program counter will match the requested address in the expression.

```
;Offset ;PC      ;Hex              ;Monitor          ;Source
                                                    *       = $0800
```

```
.0800                                           label1
                                                        .logical $1000
.0800   1000                                    label2
                                                *       = $1200
.0a00   1200                                    label3
                                                        .endlogical
.0a00                                           label4
```

.offs <expression>

Sets the compile offset relative to the program counter.

Popular in old TASM code where this was the only way to create relocated code, otherwise it's use is not recommended as there are easier to use alternatives below.

```
;Offset ;PC     ;Hex            ;Monitor        ;Source
                                                *       = $1000
.1000           ea              nop                     nop
                                                        .offs 100
.1065   1001    ea              nop                     nop
```

.logical <expression>

Starts a relocation block

.here
.endlogical

Ends a relocation block

Changes the program counter only, the compile offset is not changed. When finished all continues where it was left off before.

The naming is not logical at all for relocated code, but that's how it was named in old 6502tass.

It's used for code copied to it's proper location at runtime. Can be nested of course.

```
;Offset ;PC     ;Hex            ;Monitor        ;Source
                                                *       = $1000
                                                        .logical $300
.1000   0300     a9 80          lda #$80        drive   lda #$80
.1002   0302     85 00          sta $00                 sta $00
.1004   0304     4c 00 03       jmp $0300               jmp drive
                                                        .endlogical
```

.virtual [<expression>]

Starts a virtual block

.endv
.endvirtual

Ends a virtual block

Changes the program counter to the expression (if given) and discards the result of compilation. This is useful to define structures to fixed addresses.

```
        .virtual $d400  ; base address
sid     .block
freq    .word ?         ; frequency
pulsew  .word ?         ; pulse width
control .byte ?         ; control
ad      .byte ?         ; attack/decay
sr      .byte ?         ; sustain/release
        .endblock
        .endvirtual
```

Or to define stack "allocated" variables on 65816.

```
        .virtual #1,s
p1      .addr ?         ; at #1,s
```

```
tmp     .byte ?         ; at #3,s
        .endvirtual
        lda (p1),y      ; lda ($01,s),y
```

4.2 Aligning data or code

Alignment is about constraining data/code placement in memory.

The processor architecture doesn't have hard constraints on instruction or data placement still pages (256 bytes) come up quite often in instruction cycle times tables. Or even in errata like the indirect JMP bug which happens only if the word of the vector is crossing such page.

Other components like video chips can only display object if placed at an address divisible by 64 for example.

For code half of an address table might be spared if it's known that all the addresses have the same high bytes. Or if all interrupt routines are on the same page then it's enough to change the low byte of the vector when selecting another one.

Now it shouldn't come as a surprise that the following directives are mainly concerned about how dividing the program counter address gives a certain remainder.

The divisor in this context is called the alignment interval and is usually a number which is a power of two. Quite often 256, so that's the default.

The remainder is called offset and is by default 0. Negative offsets are a convenience feature and are internally corrected by adding the interval to it.

An interval sized memory area is called a page. It's boundary is at it's start. If data spans more than one page it's known as a page boundary cross.

Having a non-zero offset effectively shifts the boundary of a page in memory further up or down (if negative). An interval of 256 with offset of 8 gives page boundaries of `$1008`, `$1108` or `$1208` for example.

If the alignment is not good enough some alignment directives might try to correct it by adding padding. This is by default uninitialized (skip forward) but may be a fixed byte or anything more complex similarly to what the `.fill` directive accepts.

When alignment is done within named structures then it's relative to the start of the structure. This means the structure layout will always be the same independent of which address it's instantiated at. Anonymous structures do not change the way the alignment works.

The “`-Walign`” command line option can be used to emit warnings on where and how much padding was necessary for alignment.

.page `[<interval>[, <offset>]]`
: Start of page check block

.endp
.endpage
: End of page check block

This directive is a passive assertion and checks for a page difference or page crossing.

By default or with a negative interval parameter it verifies that the start and end directives are on the same page. This is what's needed to guard relative branches against jumping across pages:

```
        ldx #3
        .page           ;now this will execute
-       dex             ;in 14 cycles for sure
        bne -
        .endpage
```

With a positive size parameter it verifies that there's no page cross in the memory range between the directives. This is what's needed to guard against indexed access page cross cycle penalties:

```
*       = $10c0
        .page 256
```

```
table   .fill $40       ;table within the same page
        .endpage        ;different page here but no crossing
```

Normally a page check results in an error but the “`-Wno-error=page`” command line option can reduce it into a warning.

Once this directive reports an error it's time to rearrange the source in a way that the check passes. Or alternatively the alignment directives below can be used to avoid violating the assertion.

.align `[<interval>[, <fill>[, <offset>]]]`
Align the program counter to a page boundary

This directive is useful when code/data needs to be placed exactly to a page boundary. If that's not already the case sufficient padding is added until the next one is reached.

```
        .align $40      ;sprite bitmap (64 byte aligned)
sprite  .fill 63
        .align $400     ;screen memory (1024 byte aligned)
screen  .fill 1000
        .align $400, ?, -8;sprite pointers (last 8 bytes)
spritep .fill 8
        .align          ; page sized buffer at page boundary
sendbuf .fill 256       ;to avoid indexing penalty cycles
```

.alignblk `[<interval>[, <fill>[, <offset>]]]`
Starts alignment block.

.endalignblk
Ends alignment block.

Often the start address is not important only avoiding the page boundary matters.

This often can be achieved without any padding at all. If padding is necessary then this directive works the same as `.align` including alignment within structures.

It's typically used to place tables so that absolute indexed read accesses won't suffer page crossing cycle penalties.

```
        .alignblk       ;avoid page cross
table   .byte 0, 1, 2, 3, 4, 5, 6, 7
        .endalignblk
        lda table,x     ;no cycles wasted on access
```

In case the stronger guarantee of having both the start and the end directives in the same page is required then the alignment interval needs to be given as a negative number (e.g. −256). This may be necessary for aligning code with relative branches.

If the block size varies based on its memory location then doing the alignment may become impossible.

.alignpageind `<target>[, <interval>[, <fill>[, <offset>]]]`
Alignment of a page block indirectly.

Using `.alignblk` in the middle of executable code is usually problematic as the alignment is done there as well. This directive can do the alignment padding outside of the execution flow.

```
        rts
        .alignpageind pageblk;add alignment padding here
wait    ldx #3
pageblk .page           ;now this will execute
-       dex             ;in 14 cycles for sure
        bne -
        .endpage
```

By default and with a negative interval it tries to avoids page differences. With positive intervals page crosses. Same as the `.page` assertion block.

It is assumed that the padding inserted will move the target block as if it'd be right in front of it. If this isn't the case the alignment will fail.

If the block size varies based on its memory location then doing the alignment may become impossible.

.alignind <target>[, <interval>[, <fill>[, <offset>]]]
: Align the target location to a page boundary indirectly

This directive tries to align the target to a page boundary. If not already on one then sufficient padding will be added until the next one is reached.

```
;Align "pos" to page boundary. It must come right after "neg".
        .alignind pos
neg     .fill 8
pos     .fill 8
        .cerror (<pos) != 0, "pos should be page aligned"
        .cerror pos - neg != size(neg), "there should be no gap"
```

It is assumed that the padding inserted will move the target as if it'd be right in front of it. If this isn't the case the alignment will fail.

.fill <length>
: Usually the `.fill` directive is used to reserve space but it may be useful to do alignments as well.

```
;replacement for a .cerror overrun check and *= combo
        .fill start_address - *
;align the vectors "block" so it ends at end_address
        .fill end_address - size(vectors) - *
vectors .logical *      ;dummy non-scoped block for size()
...
;screen memory is needed but if at $9xxx then take $a000 instead
        .align $400     ;next 1024 byte alignment
        .fill (* >> 12) == $9 ? ($a000 - *) : 0
screen  .fill 1000
```

4.3 Dumping data

4.3.1 Storing numeric values

Multi byte numeric data is stored in the little-endian order, which is the natural byte order for 65xx processors. Numeric ranges are enforced depending on the directives used. Signed numbers are stored as two's complement.

When using lists or tuples their content will be used one by one. Uninitialized data ("?") creates holes of different sizes. Character string constants are converted using the current encoding.

Please note that multi character strings usually don't fit into 8 bits and therefore the `.byte` directive is not appropriate for them. Use `.text` instead which accepts strings of any length.

.byte <expression>[, <expression>, …]
: Create bytes from 8 bit unsigned constants (0–255)

.char <expression>[, <expression>, …]
: Create bytes from 8 bit signed constants (−128–127)

```
>1000  ff 03                            .byte 255, $03
>1002  41                               .byte "a"
>1003                                   .byte ?         ; reserve 1 byte
>1004  fd                               .char -3
;Store 4.4 signed fixed point constants
>1005  c8 34 32                         .char (-3.5, 3.25, 3.125) * 1p4
;Compact computed jumps using self modifying code
.1008  bd 0f 10  lda $1010,x            lda jumps,x
.100b  8d 0e 10  sta $100f              sta smod+1
```

```
.100e  d0 fe     bne $100e        smod    bne *
;Routines nearby (-128 to 127 bytes)
>1010  23 49                      jumps   .char (routine1, routine2)-smod-2
```

.word <expression>[, <expression>, …]
: Create bytes from 16 bit unsigned constants (0–65535)

.sint <expression>[, <expression>, …]
: Create bytes from 16 bit signed constants (−32768–32767)

```
>1000  42 23 55 45                        .word $2342, $4555
>1004                                     .word ?         ; reserve 2 bytes
>1006  eb fd 51 11                        .sint -533, 4433
;Store 8.8 signed fixed point constants
>100a  80 fc 40 03 20 03                  .sint (-3.5, 3.25, 3.125) * 1p8
.1010  bd 19 10  lda $1019,x              lda texts,x
.1013  bc 1a 10  ldy $101a,x              ldy texts+1,x
.1016  4c 1e ab  jmp $ab1e                jmp $ab1e
>1019  33 10 59 10               texts    .word text1, text2
```

.addr <expression>[, <expression>, …]
: Create 16 bit address constants for addresses (in current program bank)

.rta <expression>[, <expression>, …]
: Create 16 bit return address constants for addresses (in current program bank)

```
                                        *       = $12000
.012000 7c 03 20        jmp ($012003,x)         jmp (jumps,x)
>012003 50 20 32 03 92 15               jumps   .addr $12050, routine1, routine2
;Computed jumps by using stack (current bank)
                                        *       = $103000
.103000 bf 0c 30 10     lda $10300c,x           lda rets+1,x
.103004 48              pha                     pha
.103005 bf 0b 30 10     lda $10300b,x           lda rets,x
.103009 48              pha                     pha
.10300a 60              rts                     rts
>10300b ff ef a1 36 f3 42               rets    .rta $10f000, routine1, routine2
```

.long <expression>[, <expression>, …]
: Create bytes from 24 bit unsigned constants (0–16777215)

.lint <expression>[, <expression>, …]
: Create bytes from 24 bit signed constants (−8388608–8388607)

```
>1000   56 34 12                          .long $123456
>1003                                     .long ?         ; reserve 3 bytes
>1006   eb fd ff 51 11 00                 .lint -533, 4433
;Store 8.16 signed fixed point constants
>100c   5d 8f fc 66 66 03 1e 85           .lint (-3.44, 3.4, 3.52) * 1p16
>1014   03
;Computed long jumps with jump table (65816)
.1015   bd 2a 10        lda $102a,x       lda jumps,x
.1018   8d 11 03        sta $0311         sta ind
.101b   bd 2b 10        lda $102b,x       lda jumps+1,x
.101e   8d 12 03        sta $0312         sta ind+1
.1021   bd 2c 10        lda $102c,x       lda jumps+2,x
.1024   8d 13 03        sta $0313         sta ind+2
.1027   dc 11 03        jmp [$0311]       jmp [ind]
>102a   32 03 01 92 05 02        jumps    .long routine1, routine2
```

.dword <expression>[, <expression>, …]
: Create bytes from 32 bit unsigned constants (0–4294967295)

.dint <expression>[, <expression>, …]
: Create bytes from 32 bit signed constants (−2147483648–2147483647)

```
>1000   78 56 34 12             .dword $12345678
>1004                           .dword ?        ; reserve 4 bytes
>1008   5d 7a 79 e7             .dint -411469219
;Store 16.16 signed fixed point constants
>100c   5d 8f fc ff 66 66 03 00 .dint (-3.44, 3.4, 3.52) * 1p16
>1014   1e 85 03 00
```

.text `bits(<expression>[, <bit count>])`
: Create bytes from arbitrary precision unsigned and signed numbers.

.text `bytes(<expression>[, <byte count>])`
: Create bytes from arbitrary precision unsigned and signed numbers.

For cases not covered by the numeric store directives above it's possible to convert numbers to byte or bit strings and store the resulting string. If the count expression of `bytes()` and `bits()` is negative then the stored number is signed otherwise unsigned.

```
>1000   74 65 78 74 00 00 00 00 .text bytes("text", 8);pad up to 8 bytes
>1008   f4 ff ff ff ff ff ff ff .text bytes(-12, -8)  ;8 bytes signed
>1010   00 04 00 00 00 00       .text bits(1024, 48)  ;48 bits unsigned
>1016   f4 ff ff ff ff ff       .text bits(-12, -48)  ;48 bits signed
```

4.3.2 Storing string values

The following directives store strings of characters, bytes or bits as bytes. Small numeric constants can be mixed in to represent single byte control characters.

When using lists or tuples their content will be used one by one. Uninitialized data (“?”) creates byte sized holes. Character string constants are converted using the current encoding.

.text `<expression>[, <expression>, …]`
: Assemble strings into 8 bit bytes.

```
>1000   4f 45 d5  .text "oeU"
>1003   4f 45 d5  .text 'oeU'
>1006   17 33     .text 23, $33  ; bytes
>1008   0d 0a     .text $0a0d    ; $0d, $0a, little endian!
>100a   1f        .text %00011111; more bytes
```

.fill `<length>[, <fill>]`
: Reserve space (using uninitialized data), or fill with repeated bytes.

```
>1000             .fill $100      ;no fill, just reserve $100 bytes
>1100   00 00 00  .fill $4000, 0  ;16384 bytes of 0
...
>5100   55 aa 55  .fill 8000, [$55, $aa];8000 bytes of alternating $55, $aa
...
>7040   ff ff ff  .fill $8000 - *, $ff;fill up rest of EPROM with $ff
...
```

.shift `<expression>[, <expression>, …]`
: Assemble strings of 7 bit bytes and mark the last byte by setting it's most significant bit.

Any byte which already has the most significant bit set will cause an error. The last byte can't be uninitialized or missing of course.

The naming comes from old TASM and is a reference to setting the high bit of alphabetic letters which results in it's uppercase version in PETSCII.

```
.1000   a2 00          ldx #$00                ldx #0
.1002   bd 10 10       lda $1010,x      loop   lda txt,x
.1005   08             php                     php
.1006   29 7f          and #$7f                and #$7f
.1008   20 d2 ff       jsr $ffd2               jsr $ffd2
.100b   e8             inx                     inx
```

```
.100c   28            plp                     plp
.100d   10 f3         bpl $1002               bpl loop
.100f   60            rts                     rts
>1010   53 49 4e 47 4c 45 20 53      txt      .shift "single", 32, "string"
>1018   54 52 49 4e c7
```

.shiftl <expression>[, <expression>, …]

Assemble strings of 7 bit bytes shifted to the left once with the last byte's least significant bit set.

Any byte which already has the most significant bit set will cause an error as this is cut off on shifting. The last byte can't be uninitialized or missing of course.

The naming is a reference to left shifting.

```
.1000   a2 00         ldx #$00                ldx #0
.1002   bd 0d 10      lda $100d,x    loop     lda txt,x
.1005   4a            lsr a                   lsr a
.1006   9d 00 04      sta $0400,x             sta $400,x      ;screen memory
.1009   e8            inx                     inx
.100a   90 f6         bcc $1002               bcc loop
.100c   60            rts                     rts
                                              .enc "screen"
>100d   a6 92 9c 8e 98 8a 40 a6      txt      .shiftl "single", 32, "string"
>1015   a8 a4 92 9c 8f                        .enc "none"
```

.null <expression>[, <expression>, …]

Same as `.text`, but adds a zero byte to the end. An existing zero byte is an error as it'd cause a false end marker.

```
.1000   a9 07         lda #$07                lda #<txt
.1002   a0 10         ldy #$10                ldy #>txt
.1004   20 1e ab      jsr $ab1e               jsr $ab1e
>1007   53 49 4e 47 4c 45 20 53      txt      .null "single", 32, "string"
>100f   54 52 49 4e 47 00
```

.ptext <expression>[, <expression>, …]

Same as `.text`, but prepend the number of bytes in front of the string (pascal style string). Therefore it can't do more than 255 bytes.

```
.1000   a9 1d         lda #$1d                lda #<txt
.1002   a2 10         ldx #$10                ldx #>txt
.1004   20 08 10      jsr $1008               jsr print
.1007   60            rts                     rts

.1008   85 fb         sta $fb        print    sta $fb
.100a   86 fc         stx $fc                 stx $fc
.100c   a0 00         ldy #$00                ldy #0
.100e   b1 fb         lda ($fb),y             lda ($fb),y
.1010   f0 0a         beq $101c               beq null
.1012   aa            tax                     tax
.1013   c8            iny            -        iny
.1014   b1 fb         lda ($fb),y             lda ($fb),y
.1016   20 d2 ff      jsr $ffd2               jsr $ffd2
.1019   ca            dex                     dex
.101a   d0 f7         bne $1013               bne -
.101c   60            rts            null     rts
>101d   0d 53 49 4e 47 4c 45 20      txt      .ptext "single", 32, "string"
>1025   53 54 52 49 4e 47
```

4.4 Text encoding

64tass supports sources written in UTF-8, UTF-16 (be/le) and RAW 8 bit encoding. To take advantage of this capability custom encodings can be defined to map Unicode characters to 8 bit values in strings. Even in plain ASCII sources it could be useful to define escape sequences for control codes.

.enc <expression>
: Selects text encoding by a character string name or from an encoding object

Predefined encodings names are “none” and “screen” (screen code), anything else is user defined. All user encodings start without any character or escape definitions, add some as required. Please note that the encoding names are global.

This directive changes the text encoding after it therefore it's usually placed somewhere at the beginning of the source to make sure everything is covered.

While it is possible to juggle with multiple encodings throughout the source code using the .enc directive this is not recommended. For such use case .encode is better suited.

In the past the .enc directive accepted an unquoted string but currently it needs to be an expression.

```
                                .enc "screen";screen code mode
>1000   13 03 12 05 05 0e 20 03 .text "screen codes"
>1008   0f 04 05 13
.100c   c9 15     cmp #$15      cmp #"u"    ;compare screen code
                                .enc "none" ;normal mode again
.100e   c9 55     cmp #$55      cmp #"u"    ;compare PETSCII
```

.encode [<expression>]
: Encoding area start

.endencode
: Encoding area end

This directive either creates a new text encoding (if used without a parameter) or makes the one in the parameter effective within the enclosed area.

The text encoding can be assigned to a symbol in front of the directive so it can be reused whenever it's needed. This symbol can also act as a conversion function which converts a character string to a byte string using the encoding.

```
        .encode         ;starts anonymous local encoding scope
        .enc "titlefont";special character set
        .text "game title"
        .endencode      ;restores original encoding

vt100   .encode         ;define custom encoding
        .cdef " ~", 32
        .edef "{esc}", 27;add escape codes
        .edef "{moff}", [27, "[", "m"]
        .edef "{bold}", [27, "[", "1", "m"]
        .endencode

        .encode vt100   ;use custom encoding from here
        .text "{bold}bold{moff} text"
        lda #"{esc}"
        .endencode      ;restores original encoding
        cmp #vt100("{esc}");conversion when not in scope
        .enc vt100      ;select custom encoding (at start of source)
```

.cdef <start>, <end>, <coded> [, <start>, <end>, <coded>, …]
.cdef "<start><end>", <coded> [, "<start><end>", <coded>, …]
: Assigns characters in a range to single bytes.

This is a simple single character to byte translation definition. It's useful to map a range of Unicode characters to a range of bytes. The start and end positions are Unicode character codes either by numbers or by typing them. Overlapping ranges are not allowed.

```
        .enc "ascii"    ;define an ascii encoding
```

```
        .cdef " ~", 32  ;identity mapping for printable
```

.tdef <expression>, <expression> [, <expression>, <expression>, …]
Assign single characters to byte values.

Similar to `.cdef` it is a single character to byte translation definition. It's easier to use when the character codes are not consecutive. Overlapping ranges with the former and itself are not allowed.

It tries to assign Unicode character codes from the first expression to byte values from the second. More than one pair of such assignments can be given.

If the byte value expression is not iterable then it will get incremented for each character definition. This allows easy assignment of randomly scattered Unicode values to a consecutive range of bytes values.

```
        .tdef "A", 65   ;A -> 65
        .tdef "ACX", 65 ;A -> 65, C-> 66, X -> 67
        .tdef "ACX", [65, 33, 11];A -> 65, C-> 33, X -> 11
```

.edef "<escapetext>", <value> [, "<escapetext>", <value>, …]
Assigns strings to byte sequences as a translated value.

When these substrings are found in a text they are replaced by bytes defined here. When strings with common prefixes are used the longest match wins. Useful for defining non-typeable control code aliases, or as a simple tokeniser.

```
        .edef "\n", 13  ;one byte control codes
        .edef "{clr}", 147
        .edef "{crlf}", [13, 10];two byte control code
        .edef "<nothing>", [];replace with no bytes
```

The example below shows how all this fits together:

```
petscii .namespace
common  .segment;common definitions
        .cdef " @", $20;32-64 is identical
        .tdef "[£]↑←", $5b, "┼🮌│", $db
        .edef "{clr}", 147, "{cr}", 13
        .endsegment
upper   .encode;uppercase PETSCII
        #common
        .cdef "AZ", $41
        .tdef "▌▄▔▁▏▒▕🮏◤🮇├▗└┐▂┌┴┬┤▎▍🮈🮂🮃▃🭿▖▝┘▘▚", $a1
        .tdef "♠🭲🭸🭷🭶🭺🭱🭴╮╰╯🭼╲╱🭽🭾•🭻♥🭰╭╳○♣🭵♦", $c1
        .tdef "◥π🭹│", [$df, $ff, $c0, $dd]
        .endencode

lower   .encode;lowercase PETSCII
        #common
        .cdef "az", $41, "AZ", $c1;the easy ranges
        .tdef "▌▄▔▁▏▒▕🮏🮙🮇├▗└┐▂┌┴┬┤▎▍🮈🮂🮃▃✓▖▝┘▘▚", $a1
        .tdef "🮘🮕🭹│", [$df, $ff, $c0, $dd];random one to ones
        .endencode
        .endnamespace

                                .encode petscii.lower
>1000   93 d4 45 58 54 20 49 4e    .text "{clr}Text in PETSCII{cr}"
>1008   20 d0 c5 d4 d3 c3 c9 0d
                                .endencode
```

4.5 Structured data

Structures and unions can be defined to create complex data types. The offset of fields are available by using the definition's name. The fields themselves by using the instance name.

The initialization method is very similar to macro parameters, the difference is that unset parameters always return uninitialized data ("?") instead of an error.

4.5.1 Structure

Structures are for organizing sequential data, so the length of a structure is the sum of lengths of all items.

`.struct [<name>][=<default>]][, [<name>][=<default>] …]`
: Begins a structure block

`.ends [<result>][, <result> …]`
`.endstruct [<result>][, <result> …]`
: Ends a structure block

Structure definition, with named parameters and default values

`.dstruct <name>[, <initialization values>]`
`.<name> [<initialization values>]`
: Create instance of structure with initialization values

```
        .struct         ;anonymous structure
x       .byte 0         ;labels are visible
y       .byte 0         ;content compiled here
        .endstruct      ;useful inside unions

nn_s    .struct col, row;named structure
x       .byte \col      ;labels are not visible
y       .byte \row      ;no content is compiled here
        .endstruct      ;it's just a definition

nn      .dstruct nn_s, 1, 2;structure instance (within label)

        lda nn.x        ;direct field access
        ldy #nn_s.x     ;get offset of field
        lda nn,y        ;and use it indirectly

nnarray .brept 4        ;4 element "array" here
        .dstruct nn_s   ;fields directly here (without a label)
        .endrept

        lda nnarray[0].y;access of "array" field

coords2 .bfor x2, y2 in [(1,3),(4,2),(7,5)]
        .dstruct nn_s, x2, y2
        .endfor         ;initialized "array" from list
```

4.5.2 Union

Unions can be used for overlapping data as the compile offset and program counter remains the same on each line. Therefore the length of a union is the length of it's longest item.

`.union [<name>][=<default>]][, [<name>][=<default>] …]`
: Begins a union block

`.endu`
`.endunion`
: Ends a union block

Union definition, with named parameters and default values

`.dunion <name>[, <initialization values>]`
`.<name> [<initialization values>]`
: Create instance of union with initialization values

```
        .union          ;anonymous union
x       .byte 0         ;labels are visible
y       .word 0         ;content compiled here
        .endunion

nn_u    .union          ;named union
x       .byte ?         ;labels are not visible
y       .word \1        ;no content is compiled here
        .endunion       ;it's just a definition

nn      .dunion nn_u, 1 ;union instance here

        lda nn.x        ;direct field access
        ldy #nn_u.x     ;get offset of field
        lda nn,y        ;and use it indirectly
```

4.5.3 Combined use of structures and unions

The example below shows how to define structure to a binary include.

```
        .union
        .binary "pic.drp", 2
        .struct
color   .fill 1024
screen  .fill 1024
bitmap  .fill 8000
backg   .byte ?
        .endstruct
        .endunion
```

Anonymous structures and unions in combination with sections are useful for overlapping memory assignment. The example below shares zero page allocations for two separate parts of a bigger program. The common subroutine variables are assigned after in the “zp” section.

```
*       = $02
        .union          ;spare some memory
         .struct
          .dsection zp1 ;declare zp1 section
         .endstruct
         .struct
          .dsection zp2 ;declare zp2 section
         .endstruct
        .endunion
        .dsection zp    ;declare zp section
```

4.6 Macros

Macros can be used to reduce typing of frequently used source lines. Each invocation is a copy of the macro's content with parameter references replaced by the parameter texts.

`.segment [<name>][=<default>]][, [<name>][=<default>] …]`
: Start of segment block

`.endsegment [<result>][, <result> …]`
: End of segment block

  Copies the code segment as it is, so symbols can be used from outside, but this also means multiple use will result in double defines unless anonymous labels are used.

`.macro [<name>][=<default>]][, [<name>][=<default>] …]`
: Start of macro block

`.endmacro [<result>][, <result> …]`
: End of macro block

The code is enclosed in it's own block so symbols inside are non-accessible, unless a label is prefixed at the place of use, then local labels can be accessed through that label.

#<name> `[<param>][[,][<param>] …]`
.<name> `[<param>][[,][<param>] …]`
Invoke the macro after "`#`" or "`.`" with the parameters. Normally the name of the macro is used, but it can be any expression.

.endm `[<result>][, <result> …]`
Closing directive of `.macro` and `.segment` for compatibility.

```
;A simple macro
copy    .macro
        ldx #size(\1)
lp      lda \1,x
        sta \2,x
        dex
        bpl lp
        .endmacro

        #copy label, $500

;Use macro as an assembler directive
lohi    .macro
lo      .byte <(\@)
hi      .byte >(\@)
        .endmacro

var     .lohi 1234, 5678

        lda var.lo,y
        ldx var.hi,y
```

4.6.1 Parameter references

The first 9 parameters can be referenced by "`\1`"–"`\9`". The entire parameter list including separators is "`\@`".

```
name    .macro
        lda #\1         ;first parameter 23+1
        .endmacro

        #name 23+1      ;call macro
```

Parameters can be named, and it's possible to set a default value after an equal sign which is used as a replacement when the parameter is missing.

These named parameters can be referenced by `\name` or `\{name}`. Names must match completely, if unsure use the quoted name reference syntax.

```
name    .macro first, b=2, , last
        lda #\first     ;first parameter
        lda #\b         ;second parameter
        lda #\3         ;third parameter
        lda #\last      ;fourth parameter
        .endmacro

        #name 1, , 3, 4 ;call macro
```

4.6.2 Text references

In the original turbo assembler normal references are passed by value and can only appear in place of one. Text references on the other hand can appear everywhere and will work in place of e.g. quoted text or opcodes and labels. The first 9 parameters can be referenced as text by `@1`–`@9`.

```
name    .macro
        jsr print
        .null "Hello @1!";first parameter
        .endm

        #name "wth?"    ;call macro
```

4.7 Custom functions

Beyond the built-in functions mentioned earlier it's possible to define custom ones for frequently used calculations.

.sfunction `[<name>[:<expression>][=<default>], …][*<name>,] <expression>`
: Defines a simple function to return the result of a parametrised expression

.function `<name>[:<expression>][=<default>]], <name>[=<default>] …][, *<name>]`
: Defines a multi line function

.endf `[<result>][, <result> …]`
.endfunction `[<result>][, <result> …]`
: End of a multi line function

#<name> `[<param>][[,][<param>] …]`
.<name> `[<param>][[,][<param>] …]`
<name> `[<param>][[,][<param>] …]`
: Invoke a multi line function like a macro, directive or pseudo instruction

Function parameters are assigned to comma separated variable names on invocation. These variables are visible in the function scope.

Parameter values may be converted using a function whose name can be given after a colon following the variable name.

Default values may be supplied for each parameter after an equal sign. These values are calculated at function definition time only and are used when a parameter was not specified.

Extra parameters are not accepted, unless the last parameter symbol is preceded with a star, in this case these parameters are collected into a tuple.

Only those external variables and functions are available which were accessible at the place of definition, but not those at the place of invocation.

```
vicmem  .sfunction _font, _scr=0, ((_font >> 10) & $0f) | ((_scr >> 6) & $f0)

        lda #vicmem($2000, $0400); calculate constant
        sta $d018
```

If a multi line function is used in an expression only the returned result is used. If multiple values are returned these will form a tuple.

If a multi line function is used as macro, directive or pseudo instruction and there's a label in front then the returned value is assigned to it. If nothing is returned then it's used as regular label.

```
mva     .function value, target
        lda value
        sta target
        .endfunction

        mva #1, label
```

4.8 Conditional assembly

To prevent parts of source from compiling conditional constructs can be used. This is useful when multiple slightly different versions needs to be compiled from the same source.

Anonymous labels are still recognized in the non-compiling parts even if they won't get defined. This ensures consistent relative referencing across conditionally compiled areas with such labels.

4.8.1 If, else if, else

.if <condition>
: Compile if condition is true

.elsif <condition>
: Compile if previous conditions were not met and the condition is true

.else
: Compile if previous conditions were not met

.ifne <value>
: Compile if value is not zero

.ifeq <value>
: Compile if value is zero

.ifpl <value>
: Compile if value is greater or equal zero

.ifmi <value>
: Compile if value is less than zero

The `.ifne`, `.ifeq`, `.ifpl` and `.ifmi` directives exists for compatibility only, in practice it's better to use comparison operators instead.

```
        .if wait==2     ;2 cycles
        nop
        .elsif wait==3  ;3 cycles
        bit $ea
        .elsif wait==4  ;4 cycles
        bit $eaea
        .else           ;else 5 cycles
        inc $2
        .endif
```

.fi
.endif
: End of conditional compilation.

.elif <condition>
: Same as `.elsif` because it's a popular typo and it's difficult to notice.

4.8.2 Switch, case, default

Similar to the `.if`, `.elsif`, `.else`, `.endif` construct, but the compared value needs to be written only once in the switch statement.

.switch <expression>
: Evaluate expression and remember it

.case <expression>[, <expression> …]
: Compile if the previous conditions were all skipped and one of the values equals

.default
: Compile if the previous conditions were all skipped

```
        .switch wait
        .case 2         ;2 cycles
        nop
        .case 3         ;3 cycles
        bit $ea
        .case 4         ;4 cycles
        bit $eaea
        .default        ;else 5 cycles
        inc $2
        .endswitch
```

.endswitch

End of `.switch` conditional compilation block.

4.8.3 Comment

`.comment`
Never compile.

```
        .comment
        lda #1          ;this won't be compiled
        sta $d020
        .endcomment
```

`.endc`
`.endcomment`
End of `.comment` block.

4.9 Repetitions

The following directives are used to repeat code or data.

The regular non-scoped variants cover most cases except when normal labels are required as those will be double defined.

Scoped variants (those starting with the letter b) create a new scope for each iteration. This allows normal labels without collision but it's a bit more resource intensive.

If the scoped variant is prefixed with a label then the list of individual scopes for each iteration will be assigned to it. This allows accessing labels within.

`.for` `[<assignment>], [<condition>], [<assignment>]`
`.bfor` `[<assignment>], [<condition>], [<assignment>]`
Assign initial value, loop while the condition is true and modify value.

First a variable is set, usually this is used for counting. This is optional, the variable may be set already before the loop.

Then the condition is checked and the enclosed lines are compiled if it's true. If there's no condition then it's an infinite loop and `.break` must be used to terminate it.

After an iteration the second assignment is calculated, usually it's updating the loop counter variable. This is optional as well.

```
        ldx #0
        lda #32
lp      .for ue := $400, ue < $800, ue += $100
        sta ue,x         ;do $400, $500, $600 and $700
        .endfor
        dex
        bne lp
```

`.for` `<variable>[, <variable>, …]` **`in`** `<expression>[, <expression>, …]`
`.bfor` `<variable>[, <variable>, …]` **`in`** `<expression>[, <expression>, …]`
Assign variable(s) to values in sequence one-by-one in order.

Usually one variable is used to loop through all values. The values can be supplied by `range` function or some sort of list.

It's also possible to use more than one variable for each iteration. These can be assigned to a collection of values each time (row oriented) or a single value from each collection (column oriented).

A row oriented for loop expects collections of the same number of values as the number of variables as each variable gets assigned to one of them. The loop iteration count depends on how many such collections were supplied.

A column oriented for loop expects the same number of collections (comma separated) as the number of variables. On each iteration a single value is taken from each and is assigned to the matching variable. All collections should have the same length so that all variables can be assigned. This length also determines the loop iteration count.

```
;loop through on iterable or on comma separated values
        .for col in 0, 11, 12, 15, 1
        lda #col         ;0, 11, 12, 15 and 1
        sta $d020
        .endfor
;row oriented iterating for loop, on list of tuples
        .for dest, val in [($d011, $3b), ($d020, 0), ($d018, $18)]
        lda #val
        sta dest
        .endfor
;column oriented iterating for loop, one iterable for each variable.
        .for dest, val in ($d011, $d020, $d018), ($3b, 0, $18)
        lda #val
        sta dest
        .endfor
```

.endfor
: End of a `.for` or `.bfor` loop block

.rept <expression>
.brept <expression>
: Repeat enclosed lines the specified number of times.

```
        .rept 100
-       inx
        bne -
        .endrept

lst     .brept 100       ;each iteration into a tuple
label   jmp label        ;not a duplicate definition
        .endrept
        jmp lst[5].label ;use label of 6th iteration
```

.endrept
: End of a `.rept` or `.brept` block

.while <condition>
.bwhile <condition>
: Repeat enclosed lines until the condition holds.

Works as expected however the scoped variant might be tricky to use as variables of the condition are usually part of the parent scope. So modifying them in the loop body should be done with compound assignments or the reassign operator (`::=`).

.endwhile
: End of a `.while` or `.bwhile` loop block

.break
: Exit current repetition loop immediately.

.breakif <condition>
: Exit current repetition loop immediately if the condition holds.

It's a shorthand for a `.if`, `.break`, `.endif` sequence.

.continue
: Continue current repetition loop's next iteration.

.continueif <condition>
: Continue current repetition loop's next iteration if the condition holds.

It's a shorthand for a `.if`, `.continue`, `.endif` sequence.

.next
: Closing directive of `.for`, `.bfor`, `.rept`, `.brept`, `.while` and `.bwhile` loop for compatibility.

.lbl
Creates a special jump label that can be referenced by `.goto`

.goto `<labelname>`
Causes assembler to continue assembling from the jump label. No forward references of course, handle with care. Should only be used in classic TASM sources for creating loops.

```
i       .var 100
loop    .lbl
        nop
i       .var i - 1
        .ifne i
        .goto loop      ;generates 100 nops
        .endif          ;the hard way ;)
```

4.10 Including files

Longer sources are usually separated into multiple files for easier handling. Precomputed binary data can also be included directly without converting it into source code first.

Search path is relative to the location of current source file. If it's not found there the include search path is consulted for further possible locations.

To make your sources portable please always use forward slashes (`/`) as a directory separator and use lower/uppercase consistently in file names!

.include `<filename>`
Include source file here.

.binclude `<filename>`
Include source file here in it's local block. If the directive is prefixed with a label then all labels are local and are accessible through that label only, otherwise not reachable at all.

```

        .include "macros.asm"       ;include macros
menu    .binclude "menu.asm"        ;include in a block
        jmp menu.start
```

.binary `<filename>[, <offset>[, <length>]]`
Include raw binary data from file.

By using offset and length it's possible to break out chunks of data from a file separately, like bitmap and colors for example. Negative offsets are calculated from the end of file.

```
        .binary "stuffz.bin"        ;simple include, all bytes
        .binary "stuffz.bin", 2     ;skip start address
        .binary "stuffz.bin", 2, 1000;skip start address, 1000 bytes max
```

4.11 Scopes

Scopes may contain symbols or further nested scopes. The same symbol name can be reused as long as it's in a different scope.

A symbol is looked up in the local scope first. If it's a non-local symbol then parent scopes and the global scope may be searched in addition. This means that a symbol in a parent or global scope may be “shadowed”.

Symbols of a named scope can be looked up using the “`.`” operator. The searched symbol stands on the right and it's looked up in the scope on the left. More than one symbol may be looked up at the same time and the result will be a list or tuple.

```
        lda #0
        sta vic.sprite.enable
; same as .byte colors.red, colors.green, colors.blue
ctable  .byte colors.(red, green, blue)
```

.proc

Start of a procedure block

.pend
.endproc
End of a procedure block

If the symbol in front is not referenced anywhere then the enclosed source won't be compiled.

Symbols inside are enclosed in a scope and are accessible through the symbol of the procedure using the dot notation. This forces compilation of the whole procedure of course.

```
ize     .proc
        nop
cucc    nop
        .endproc

        jsr ize
        jmp ize.cucc
```

The “compilation only if used” behaviour of `.proc` eases the building of “libraries” from a collection of subroutines and tables where not everything is needed all the time.

Alternative dead-code reduction techniques I encountered:

Separate source files
: This potentially results in a lot of small files and manually managed include directives. This is popular on external linker based systems where object files may be excluded if unused.

Conditional compilation
: Few larger files with conditional compilation directives all over the place to exclude or include various parts. The source which does the include manually declares somewhere what's actually needed or not. There may be a lot of options if it's fine grained enough.

Wrap parts with macros
: If a part is needed then a single macro call is placed somewhere to “include” that part. Much better than conditional compilation but these macro calls still need to be manually managed.

.block
Block scoping area start

.bend
.endblock
Block scoping area end

All symbols inside a block are enclosed in a scope. If the block had a symbol then local symbols are accessible through that using the dot notation.

```
        .block
        inc count + 1
count   ldx #0
        .endblock
```

.namespace [<expression>]
Namespace area start

.endn
.endnamespace
Namespace area end

This directive either creates a new scope (if used without a parameter) or activates the one in the parameter.

The scope can be assigned to a symbol in front of the directive so that it can be reactivated later. This enables label definitions into the same scope in different files.

```
colors  .namespace
red     = 2
blue    = 6
```

```
        .endnamespace
        lda #colors.red
```

.weak
.endweak

Weak symbol area

Any symbols defined inside can be overridden by "stronger" symbols in the same scope from outside. Can be nested as necessary.

This gives the possibility of giving default values for symbols which might not always exist without resorting to `.ifdef`/`.ifndef` or similar directives in other assemblers.

```
symbol  = 1             ;stronger symbol than the one below
        .weak
symbol  = 0             ;default value if the one above does not exists
        .endweak
        .if symbol      ;almost like an .ifdef ;)
```

Other use of weak symbols might be in included libraries to change default values or replace stub functions and data structures.

If these stubs are defined using `.proc`/`.endproc` then their default implementations will not even exists in the output at all when a stronger symbol overrides them.

Multiple definition of a symbol with the same "strength" in the same scope is of course not allowed and it results in double definition error.

Please note that `.ifdef`/`.ifndef` directives are left out from 64tass for of technical reasons, so don't wait for them to appear anytime soon.

.with `<expression>`
.endwith

Namespace access

This directive is similar to `.namespace` but it gives access to another scope's variables without leaving the current scope. May be useful to allow a short hand access in some situations.

It's advised to use the "`-Wshadow`" command line option to warn about any unexpected symbol ambiguity.

4.12 Sections

Sections can be used to collect data or code into separate memory areas without moving source code lines around. This is achieved by having separate compile offset and program counters for each defined section.

.section `<name>`

Starts a segment block

.send `[<name>]`
.endsection `[<name>]`

Ends a segment block

Defines a section fragment. The name at `.endsection` must match but it's optional.

.dsection `<name>`

Collect the section fragments here.

All `.section` fragments are compiled to the memory area allocated by the `.dsection` directive. Compilation happens as the code appears, this directive only assigns enough space to hold all the content in the section fragments.

The space used by section fragments is calculated from the difference of starting compile offset and the maximum compile offset reached. It is possible to manipulate the compile offset in fragments, but putting code before the start of `.dsection` is not allowed.

```
*       = $02
```

```
        .dsection zp   ;declare zero page section
        .cerror * > $30, "Too many zero page variables"

*       = $334
        .dsection bss   ;declare uninitialized variable section
        .cerror * > $400, "Too many variables"

*       = $0801
        .dsection code   ;declare code section
        .cerror * > $1000, "Program too long!"

*       = $1000
        .dsection data   ;declare data section
        .cerror * > $2000, "Data too long!"
;--------------------
        .section code
        .word ss, 2005
        .null $9e, format("%4d", start)
ss      .word 0

start   sei
        .section zp     ;declare some new zero page variables
p2      .addr ?         ;a pointer
        .endsection zp
        .section bss    ;new variables
buffer  .fill 10        ;temporary area
        .endsection bss

        lda (p2),y
        lda #<label
        ldy #>label
        jsr print

        .section data   ;some data
label   .null "message"
        .endsection data

        jmp error
        .section zp     ;declare some more zero page variables
p3      .addr ?         ;a pointer
        .endsection zp
        .endsection code
```

The compiled code will look like:

```
>0801    0b 08 d5 07                            .word ss, 2005
>0805    9e 32 30 36 31 00                      .null $9e, format("%4d", start)
>080b    00 00                          ss      .word 0

.080d    78                             start   sei

>0002                                   p2      .addr ?         ;a pointer
>0334                                   buffer  .fill 10        ;temporary area

.080e    b1 02                                  lda (p2),y
.0810    a9 00                                  lda #<label
.0812    a0 10                                  ldy #>label
.0814    20 1e ab                               jsr print

>1000    6d 65 73 73 61 67 65 00        label   .null "message"

.0817    4c e2 fc                               jmp error

>0004                                   p2      .addr ?         ;a pointer
```

Sections can form a hierarchy by nesting a `.dsection` into another section. The section names must only be unique within a section but can be reused otherwise. Parent section names are visible for children, siblings can be reached through parents.

In the following example the included sources don't have to know which “code” and “data” sections they use, while the “bss” section is shared for all banks.

```
;First 8K bank at the beginning, PC at $8000
*       = $0000
        .logical $8000
        .dsection bank1
        .cerror * > $a000, "Bank1 too long"
        .endlogical

bank1   .block          ;Make all symbols local
        .section bank1
        .dsection code  ;Code and data sections in bank1
        .dsection data
        .section code   ;Pre-open code section
        .include "code.asm"; see below
        .include "iter.asm"
        .endsection code
        .endsection bank1
        .endblock

;Second 8K bank at $2000, PC at $8000
*       = $2000
        .logical $8000
        .dsection bank2
        .cerror * > $a000, "Bank2 too long"
        .endlogical

bank2   .block          ;Make all symbols local
        .section bank2
        .dsection code  ;Code and data sections in bank2
        .dsection data
        .section code   ;Pre-open code section
        .include "scr.asm"
        .endsection code
        .endsection bank2
        .endblock

;Common data, avoid initialized variables here!
*       = $c000
        .dsection bss
        .cerror * > $d000, "Too much common data"
;------------- The following is in "code.asm"
code    sei

        .section bss   ;Common data section
buffer  .fill 10
        .endsection bss

        .section data  ;Data section (in bank1)
routine .addr print
        .endsection bss
```

4.13 65816 related

.as
.al

Select short (8 bit) or long (16 bit) accumulator immediate constants.

```
        .al
```

```
        lda #$4322
```

.xs
.xl
Select short (8 bit) or long (16 bit) index register immediate constants.

```
        .xl
        ldx #$1000
```

.autsiz
.mansiz
Select automatic adjustment of immediate constant sizes based on SEP/REP instructions.

```
        .autsiz
        rep #$10        ;implicit .xl
        ldx #$1000
```

.databank <expression>
Data bank (absolute) addressing is only used for addresses falling into this 64 KiB bank. The default is 0, which means addresses in bank zero.

When data bank is switched off only data bank indexed (,b) addresses create data bank accessing instructions.

```
        .databank $10   ;data bank at $10xxxx
        lda $101234     ;results in $ad, $34, $12
        .databank ?     ;no data bank
        lda $1234       ;direct page or long addressing
        lda #$1234,b    ;results in $ad, $34, $12
```

.dpage <expression>
Direct (zero) page addressing is only used for addresses falling into a specific 256 byte address range. The default is 0, which is the first page of bank zero.

When direct page is switched off only the direct page indexed (,d) addresses create direct page accessing instructions.

```
        .dpage $400     ;direct page $400-$4ff
        lda $456        ;results in $a5, $56
        .dpage ?        ;no direct page
        lda $56         ;data bank or long addressing
        lda #$56,d      ;results in $a5, $56
```

4.14 Controlling errors

.option allow_branch_across_page
Switches error generation on page boundary crossing during relative branch. Such a condition on 6502 adds 1 extra cycle to the execution time, which can ruin the timing of a carefully cycle counted code.

```
        .option allow_branch_across_page = 0
        bcc +           ;same execution time
        inx             ;needed in both cases
+       bcs +
        dex
+       .option allow_branch_across_page = 1
```

.error <message> [, <message>, …]
.cerror <condition>, <message> [, <message>, …]
Exit with error or conditionally exit with error

```
        .error "Unfinished here..."
        .cerror * > $1200, "Program too long by ", * - $1200, " bytes"
```

.warn <message> [, <message>, …]
.cwarn <condition>, <message> [, <message>, …]
: Display a warning message always or depending on a condition

```
        .warn "FIXME: handle negative values too!"
        .cwarn * > $1200, "This may not work!"
```

4.15 Target

.cpu <expression>
: Selects CPU according to the string argument.

In the past the `.cpu` directive accepted an unquoted string but currently it needs to be an expression.

```
        .cpu "6502"      ;standard 65xx
        .cpu "65c02"     ;CMOS 65C02
        .cpu "65ce02"    ;CSG 65CE02
        .cpu "6502i"     ;NMOS 65xx
        .cpu "65816"     ;W65C816
        .cpu "65dtv02"   ;65dtv02
        .cpu "65el02"    ;65el02
        .cpu "r65c02"    ;R65C02
        .cpu "w65c02"    ;W65C02
        .cpu "4510"      ;CSG 4510
        .cpu "45gs02"    ;45GS02
        .cpu "default"   ;cpu set on command line
```

4.16 Misc

.end
: Terminate assembly. Any content after this directive is ignored.

.eor <expression>
: XOR output with an 8 bit value. Useful for reverse screen code text for example, or for silly “encryption”.

.seed <expression>
: Seed the pseudo random number generator with an unsigned integer of maximum 128 bits to make the generated numbers less boring.

.var <expression>
: Defines a variable identified by the label preceding, which is set to the value of expression or reference of variable.

```
counter .var 0              ;define, same as :=
counter .var counter + 1    ;redefine, same as += 1
```

.from <scope>
: Defines a symbol to the value of the same symbol from another scope.

This directive looks up the symbol name to be defined in the other scope for its value. Useful for shorthand definitions without repeating the name if it's unchanged.

```
red     .from vic.colors    ;same as red = vic.colors.red

init    .from +             ;expose these symbols publicly
play    .from +
+       .block              ;other symbols hidden in block
init    sei
...
```

.assert
.check
: Do not use these, the syntax will change in next version!

4.17 Printer control

`.pron`
`.proff`
Turn on or off source listing on part of the file.

```
        .proff           ;Don't put filler bytes into listing
*       = $8000
        .fill $2000, $ff ;Pre-fill ROM area
        .pron
*       = $8000
        .addr reset, restore
        .text "CBM80"
reset   cld
```

`.hidemac`
`.showmac`
Ignored for compatibility.

5 Pseudo instructions

5.1 Aliases

For better code readability `BCC` has an alias named `BLT` (**B**ranch **L**ess **T**han) and `BCS` one named `BGE` (**B**ranch **G**reater **E**qual).

```
        cmp #3
        blt exit        ; less than 3?
```

For similar reasons `ASL` has an alias named `SHL` (**SH**ift **L**eft) and `LSR` one named `SHR` (**SH**ift **R**ight). This naming however is not very common.

The implied variants `LSR`, `ROR`, `ASL` and `ROL` are a shorthand for `LSR A`, `ROR A`, `ASL A` and `ROL A`. Using the implied form is considered poor coding style.

For compatibility `INA` and `DEA` are a shorthand of `INC A` and `DEC A`. Therefore there's no “implied” variants like `INC` or `DEC`. The full form with the accumulator is preferred.

Many illegal opcodes have aliases for compatibility as there's no standard naming convention.

5.2 Generic instructions

Instructions operating on different registers have different mnemonics, for example `CMP` and `CPX`. That makes writing code fast and easy but sometimes it'd be nice to parametrize the register as well not just the address. It's not like it's unsolvable through conditional compilation directives or macro text replacement but this is an alternative.

The processor registers have predefined single letter symbols like `A`, `X` or `Y`. These can be assigned to symbols with longer names if needed.

Parametrizing register increment and decrement instructions is straightforward by using the register as the address similar like `ASL A`:

DEA	DEX	DEY	DEZ	INA	INX	INY	INZ
DEC A	DEC X	DEC Y	DEC Z	INC A	INC X	INC Y	INC Z

Table 28: Register addressing modes of increment and decrement

Push and pull instructions have a generic version called `PSH` and `PUL`. The user stack ones for 65EL02 are called `RSH` and `RUL`. These take a register address:

PHA	PHB	PHD	PHK	PHP	PHX	PHY	PHZ
PSH A	PSH B	PSH D	PSH K	PSH P	PSH X	PSH Y	PSH Z
PLA	PLB	PLD		PLP	PLX	PLY	PLZ
PUL A	PUL B	PUL D		PUL P	PUL X	PUL Y	PUL Z

Table 29: Register parametrized push and pull instructions

RHA	RHI	RHX	RHY	RLA	RLI	RLX	RLY
RSH A	RSH I	RSH X	RSH Y	RUL A	RUL I	RUL X	RUL Y

The parametrized version of load and store instructions are `LDR` and `STR`, respectively. The first parameter is the register to be used and then the regular addressing mode can be written after a coma. For example `LDX $1000,Y` is equivalent to `LDR X, $1000,Y`. It is recommended to leave a space before the addressing mode.

LDA	LDX	LDY	LDZ	STA	STX	STY	STZ
LDR A,	LDR X,	LDR Y,	LDR Z,	STR A,	STR X,	STR Y,	STR Z,
CPA	CPX	CPY	CPZ		LDQ	STQ	CPQ
CMP A,	CMP X,	CMP Y,	CMP Z,		LDR Q,	STR Q,	CMP Q,

Table 30: Register parametrized load, store and compare instructions

Transfer instructions have generic versions as well, actually in two variants. The first one is when only one of the registers need to be variable. A register address can be used with regular load instructions to get those setting the flags, like `TAX` which is `LDA X`. Store instructions are used for those which don't, like `TXS` which is `STX S`.

TBA	TDC	TSC	TXA	TYA	TZA		
LDA B	LDA D	LDA S	LDA X	LDA Y	LDA Z		
	TAX	TIX	TRX	TSX	TYX	TAY	TSY
	LDX A	LDX I	LDX R	LDX S	LDX Y	LDY A	LDY S
TXY	TAZ			TAB	TCS	TXS	TYS
LDY X	LDZ A			STA B	STA S	STX S	STY S

Table 31: Register transfer parametrization through load and stores

The second variant allows parametrizing both the source and target register. This can be done with `LDR` for those setting the flags and `STR` for those which don't. Here it's especially important to leave a space after the coma to not create an unintended register indexing mode.

TBA	TDC	TSC	TXA	TYA	TZA	TCD	TXI
LDR A, B	LDR A, D	LDR A, S	LDR A, X	LDR A, Y	LDR A, Z	LDR D, A	LDR I, X
TXR	TAX	TIX	TRX	TSX	TYX	TAY	TSY
LDR R, X	LDR X, A	LDR X, I	LDR X, R	LDR X, S	LDR X, Y	LDR Y, A	LDR Y, S
TXY	TAZ			TAB	TCS	TXS	TYS
LDR Y, X	LDR Z, A			STR A, B	STR A, S	STR X, S	STR Y, S

Table 32: Register transfer fully parametrized form

In the following subroutine has input parameters in registers. These were given symbols and are used to load them at the call site using `LDR`. The pointer address is a pair of registers in a tuple and is stored with `STR`. The the `Y` register was named as pos to show that it's possible even if it's not very useful in this case.

```
=$ffd2                                          out     = $ffd2

                                                *       = $1000

.1000                                           print   .proc
=$02                                            txtp    = $02
=register("x")                                  length  = x
=(register("y"),register("a"))                  txtadr  = (y, a)
=register("y")                                  pos     = y

.1000   84 02           sty $02                         str txtadr, txtp+(0, 1)
.1002   85 03           sta $03
.1004   a0 00           ldy #$00                        ldr pos, #0
.1006   b1 02           lda ($02),y             -       lda (txtp),pos
.1008   20 d2 ff        jsr $ffd2                       jsr out
.100b   c8              iny                             inc pos
.100c   ca              dex                             dec length
.100d   d0 f7           bne $1006                       bne -
.100f   60              rts                             rts
```

```
                                             .pend

>1011   52 45 41 44 59 2e           text     .text "ready."

.1017   a0 11        ldy #$11                ldr print.txtadr, #(<text, >text)
.1019   a9 10        lda #$10
.101b   a2 06        ldx #$06                ldr print.length, #size(text)
.101d   20 00 10     jsr $1000               jsr print
```

A simpler example to push and pull multiple registers on a 65C02:

```
irq     psh (a, x, y,)  ;pha, phx, phy
        ...
        pul (y, x, a,)  ;ply, plx, pla
        rti
```

5.3 Always taken branches

Special pseudo instructions exist for using shorter conditional branches in place of longer jump instructions. Their names are derived from conditional branches and are: `GEQ`, `GNE`, `GCC`, `GCS`, `GPL`, `GMI`, `GVC`, `GVS`, `GLT` and `GGE`. Arranging the flags so that these branch are always taken is the responsibility of the programmer of course.

The main use case is that these 2 byte branches automatically turn into a 3 byte `JMP` or `BRL` once the target gets too far.

```
.0000   a9 03        lda #$03    in1     lda #3       ;not zero
.0002   d0 02        bne $0006           gne at       ;branch always
.0004   a9 02        lda #$02    in2     lda #2
.0006   4c 00 10     jmp $1000   at      gne $1000    ;branch farther
```

If the branch destination is a `RTS`, `RTI` or `RTL` instruction then this opcode will used in place of the branch to save space. This only works if the destination label is on the same line as the return instruction.

```
.1000   60           rts                 gne exit     ;just return
.1001   ea           nop                 nop
.1002   60           rts         exit    rts
```

There's special support for short forward branches. These only work with labels or anonymous labels as `*+2` or hardcoded numbers would cause a contradiction.

If the branch pseudo instructions would not skip anything at all then no code is generated. This is useful for “fall-through” cases.

```
.000c                                    geq at2      ;dummy branch
.000c   ea           nop         at2     nop
```

If the branch would skip only one byte then the opposite condition is compiled (turns it to never taken) and only a single opcode byte is emitted. As the following byte becomes part of the instruction it gets skipped.

```
.0009   18           clc         in3     clc
.000a   b0           bcs                 gcc +        ;one byte skip
.000b   38           sec         in4     sec          ;sec is skipped!
.000c   ea           nop         +       nop
```

If the CPU has long conditional branches (65CE02 / 4510 / 45GS02) then the same trick above is applied to produce two byte skips as well.

There's a pseudo opcode called `GRA` for CPUs having `BRA` which can turn into a `BRL` (if available) or `JMP` for longer distances. The condition can't be reversed for a one byte skip but if the CPU has a `NOP` immediate instruction (R65C02 / W65C02) then that'll be used for this special case.

```
.1000   82           nop #               gra skip     ;on r65c02
```

```
.1001   60          rts             rts
.1002                         skip
```

5.4 Long branches

Conditional branches usually have limited range. This limitation can be worked around by jumping over a JMP or BRL instruction with the opposite condition.

To avoid hardcoding potentially unnecessary long branches 64tass can apply this trick automatically when necessary. This can be enabled with the "--long-branch" option.

```
.0000   ea          nop             nop
.0001   b0 03       bcs $0006       bcc $1000       ;long branch
.0003   4c 00 10    jmp $1000
.0006   1f 17 03    bbr 1,$17,$000c bbs 1,23,$1000  ;on R65C02
.0009   4c 00 10    jmp $1000
.000c   30 03       bmi $0011       bpl $1000       ;on 65816
.000e   82 e9 1f    brl $1000
.0011   ea          nop             nop
```

In the listing above an extra JMP instructions is shown for each long branch. In practice that's not really the case as the assembler can avoid redundant JMP instructions if several long branches target the same destination. But for that to work a numeric address match alone isn't sufficient.

```
.1000   b0 03       bcs $1005       bcc target
.1002   4c 00 20    jmp $2000
.1003   ea          nop             nop
.1006   d0 fb       bne $1002       bne target       ;jump reused
.2000                         target
```

If the branch destination is a RTS, RTI or RTL instruction then this opcode will used in place of the jump instruction to save space. This only works if the destination label is on the same line as the return instruction.

```
.1000   b0 01       bcs $1003       bcc exit        ;uses rts
.1002   60          rts
.1003   ea          nop             nop
.2000   60          rts       exit  rts
```

It's possible to figure out without reading the listing file or disabling the long branch function which branches were turned into long ones by using the "-Wlong-branch" command line option.

Forcing address size using @b on a branch disables the automatic long branch function there.

6 Original turbo assembler compatibility

6.1 How to convert source code for use with 64tass

Currently there are two options, either use "TMPview" by Style to convert the source file directly, or do the following:

- load turbo assembler, start (by SYS 9*4096 or SYS 8*4096 depending on version)
- ← then l to load a source file
- ← then w to write a source file in PETSCII format
- convert the result to ASCII using petcat (from the vice package)

The resulting file should then (with the restrictions below) assemble using the following command line:

```
64tass -C -T -a -W -i source.asm -o outfile.prg
```

6.2 Differences to the original turbo ass macro on the C64

64tass is nearly 100% compatible with the original "Turbo Assembler", and supports most of the features of the original "Turbo Assembler Macro". The remaining notable differences are listed here.

6.3 Labels

The original turbo assembler uses case sensitive labels, use the "`--case-sensitive`" command line option to enable this behaviour.

6.4 Expression evaluation

There are a few differences which can be worked around by the "`--tasm-compatible`" command line option. These are:

The original expression parser has no operator precedence, but 64tass has. That means that you will have to fix expressions using braces accordingly, for example `1+2*3` becomes `(1+2)*3`.

The following operators used by the original Turbo Assembler are different:

`.`	bitwise or, now `|`
`:`	bitwise eor, now `^`
`!`	force 16 bit address, now `@w`

Table 33: TASM Operator differences

The default expression evaluation is not limited to 16 bit unsigned numbers anymore.

6.5 Macros

Macro parameters are referenced by "`\1`"–"`\9`" instead of using the pound sign.

Parameters are always copied as text into the macro and not passed by value as the original turbo assembler does, which sometimes may lead to unexpected behaviour. You may need to make use of braces around arguments and/or references to fix this.

6.6 Bugs

Some versions of the original turbo assembler had bugs that are not reproduced by 64tass, you will have to fix the code instead.

In some versions labels used in the first `.block` are globally available. If you get a related error move the respective label out of the `.block`.

7 Command line options

Short command line options consist of "`-`" and a letter, long options start with "`--`".

If "`--`" is encountered then further options are not recognized and are assumed to be file names.

Options requiring file names are marked with "<filename>". A single "`-`" as name means standard input or output. File name quoting is system specific.

7.1 Output options

-o `<filename>`, **--output** `<filename>`

Place output into <filename>. The default output filename is "`a.out`". This option changes it.

```
64tass a.asm -o a.prg
```

This option may be used multiple times to output different sections in different formats of a single compilation.

For multiple outputs the format options and output section selection must be placed before this option. The format selection will be unchanged if no new selection was made but the output section selection and the map file must be repeated for each output. The maximum image size will be the smallest of all selected formats. Using the same name multiple times is not a good idea.

--output-append <filename>
: Same as the `--output` option but appends instead of overwrites.

  Normally output files are overwritten but in some cases it's useful to append them instead.

--no-output
: No output file will be written.

  Useful for test compiles.

--map <file>
: Specify memory map output file.

  Normally the memory map is displayed on the standard output together with other messages. It's possible to write it to a file or to the standard output by using “-” as the file name.

--map-append <filename>
: Same as the `--map` option but appends instead of overwrites.

--no-map
: Do not display or record the memory map.

--output-section <sectionname>
: Specify which section to write in the output.

  By default all sections go into the output file. Using this option limits the output to specific section and it's children. This is useful to split a larger program into several files.

```
64tass a.asm --output-section main -o main.prg \
             --output-section loader -o loader.prg
```

--output-exec <expression>
: Sets execution address for output formats which support this.

  While it's possible to enter the address as a number it's recommended to use a label instead.

-X, **--long-address**
: Use 3 byte address/length for CBM and nonlinear output instead of 2 bytes. Also increases the size of raw output to 16 MiB and prevent the use of S19 for S-record.

```
64tass --long-address --m65816 a.asm
```

--cbm-prg
: Generate CBM format binaries (default)

  Overlapping blocks are flattened and uninitialized memory is filled up with zeros. Uninitialized memory before the first and after the last valid bytes are not saved. Up to 64 KiB normally or 16 MiB with the `--long-address` command line parameter.

  Used for C64 binaries. The first 2 bytes are the little endian address of the first valid byte (load address). This is followed by the data.

```
64tass --cbm-prg a.asm
*       = $2000
start   rts
```

`00 20`	load to $2000
`60`	data

Table 34: Example CBM format binary output

-b, **--nostart**
: Output raw binary data.

  Overlapping blocks are flattened and uninitialized memory is filled up with zeros. Uninitialized memory before the first and after the last valid bytes are not saved. Up to 64 KiB normally or 16 MiB with the `--long-address` command line parameter.

  Useful for small ROM files.

```
64tass --nostart a.asm
*       = $2000
        rts
```

60	data

Table 35: Example raw output

-f, --flat
Flat address space output mode.

Overlapping blocks are flattened and uninitialized memory is filled up with zeros. Uninitialized memory after the last valid byte is not saved. Up to 4 GiB.

Useful for creating huge multi bank ROM files. See sections for an example.

-n, --nonlinear
Generate nonlinear output file.

Overlapping blocks are flattened. Blocks are saved in sorted order and uninitialized memory is skipped. Up to 64 KiB normally or 16 MiB with the `--long-address` command line parameter.

Used for linkers and downloading. Before writing each memory block the length and the memory address is saved in a little endian order. Once everything was saved a zero length block is written without an address or data. These zeros serve as an end marker.

```
64tass --nonlinear a.asm
*       = $1000         ;1st segment
        lda #2
*       = $2000         ;2nd segment
        rts
```

02 00	load 2 bytes
00 10	to $1000
a9 02	data
01 00	load 1 byte
00 20	to $2000
60	data
00 00	load 0 bytes, end marker

Table 36: Example 64 KiB nonlinear output

--atari-xex
Generate an Atari XEX output file.

Overlapping blocks are kept, continuing blocks are concatenated. Saving happens in the definition order without sorting, and uninitialized memory is skipped in the output. Up to 64 KiB.

Used for Atari executables. First 2 bytes of signature is written. Then before saving each memory block the words of load address and last byte address is written in little endian format.

If the `--output-exec` command line parameter was given then a 6 byte run block is added to the end of the output.

```
64tass --output-exec=start --atari-xex a.asm
*       = $2000
start   rts
```

ff ff	header
00 20	load to $2000
00 20	until $2000
60	data
e0 02 e1 02	run marker
00 20	run address ($2000)

Table 37: Example Atari XEX format output

--apple-ii

Generate an Apple II output file (DOS 3.3).

Overlapping blocks are flattened and uninitialized memory is filled up with zeros. Uninitialized memory before the first and after the last valid bytes are not saved. Up to 64 KiB.

Used for Apple II executables. First the load address and the data length words are written in little endian format. This is followed by the data.

```
64tass --apple-ii a.asm
*       = $0c00
        rts
```

00 0c	load to $0c00
01 00	length is 1 byte
60	data

Table 38: Example of Apple II format output

--c256-pgx

Generate C256 Foenix PGX output file.

Overlapping blocks are flattened and uninitialized memory is filled up with zeros. Uninitialized memory before the first and after the last valid bytes are not saved. Up to 16 MiB.

Used for single segment C256 Foenix executables. After the PGX signature a four byte little endian load address is written. This is followed by the data.

```
64tass --c256-pgx a.asm
*       = $1000
        rts
```

50 47 58 01	PGX signature
00 10 00 00	load to $1000
60	data

Table 39: Example PGX format output

--c256-pgz

Generate C256 Foenix PGZ output file.

Overlapping blocks are flattened. Blocks are saved in sorted order and uninitialized memory is skipped. Up to 16 MiB.

Used for multi segment C256 Foenix binaries. It starts with a single byte signature. Then before each memory block a three byte load address and length is written in little endian format.

If the `--output-exec` command line parameter was given then a 6 byte execution block is added to the end of the output.

```
64tass --output-exec=start --c256-pgz a.asm
*       = $1000
start   rts
```

5a	PGZ signature byte
00 10 00	load to $1000
01 00 00	length is 1 bytes
60	data
00 10 00	execute at $1000
00 00 00	execution marker

Table 40: Example PGZ format output

--cody-bin

Generate a Cody binary output file

Overlapping blocks are flattened and uninitialized memory is filled up with zeros. Uninitialized memory before the first and after the last valid bytes are not saved. Up to

64 KiB.

Used for Cody executables. First the load address and last byte's address words are written in little endian format. This is followed by the data.

```
64tass --cody-bin a.asm
*       = $6300
        nop
        rts
```

`00 63`	load to `$6300`
`01 63`	last byte
`EA 60`	data

Table 41: Example of Cody format output

`--wdc-bin`

Generate WDC binary output file. Also known as zardoz binary.

Overlapping blocks are flattened. Blocks are saved in sorted order and uninitialized memory is skipped. Up to 16 MiB.

It starts with a single byte signature. Then before each memory block a three byte load address and length is written in little endian format. Ends with a zero load and start address.

```
64tass --output-exec=start --wdc-bin a.asm
*       = $1000
start   rts
```

`5a`	Z signature byte
`00 10 00`	load to `$1000`
`01 00 00`	length is 1 bytes
`60`	data
`00 00 00`	end
`00 00 00`	marker

Table 42: Example WDC binary (zardoz) format output

`--intel-hex`

Use Intel HEX output file format.

Overlapping blocks are kept, data is stored in the definition order, and uninitialized areas are skipped. I8HEX up to 64 KiB, I32HEX up to 4 GiB.

Used for EPROM programming or downloading. Data bytes are written using `00` records. If the file is larger than 64 KiB then `04` records are used as needed. The output ends with a `01` record.

If the `--output-exec` command line parameter was given then a `05` record is added with the execution address right before the end `01` record.

```
64tass --intel-hex a.asm
*       = $0c00
        rts
```

Example Intel HEX output:

```
:010C00006093
:00000001FF
```

`--mos-hex`

Use MOS Technology output file format. Also known as Paper Tape Format.

Overlapping blocks are kept, data is stored in the definition order, and uninitialized areas are skipped. Up to 64 KiB.

```
64tass --mos-hex a.asm
*       = $0c00
```

```
        rts
```

Example MOS Technology output:

```
;010C0060006D
;0000010001
```

--s-record
Use Motorola S-record output file format.

Overlapping blocks are kept, data is stored in the definition order, and uninitialized memory areas are skipped. S19 up to 64 KiB, S28 up to 16 MiB and S37 up to 4 GiB.

Used for EPROM programming or downloading. First a `S0` header record is written which is followed by `S1`, `S2`, or `S3` data records. Then an `S5` or `S6` count record comes and a `S9`, `S8` or `S7` termination record.

If the `--output-exec` command line parameter was given then the termination record will use this address. Without this the address of the first data record is used.

The `--long-address` command line parameter can be used to avoid the use of S19 format.

```
64tass --s-record a.asm
*       = $0c00
        rts
```

Example Motorola S-record output:

```
S00600004844521B
S1040C00608F
S5030001FB
S9030C00F0
```

7.2 Operation options

-a, --ascii
Use ASCII/Unicode text encoding instead of raw 8-bit

Normally no conversion takes place, this is for backwards compatibility with a DOS based Turbo Assembler editor, which could create PETSCII files for 6502tass. (including control characters of course)

Using this option will change the default “none” and “screen” encodings to map `'a'-'z'` and `'A'-'Z'` into the correct PETSCII range of `$41-$5A` and `$C1-$DA`, which is more suitable for an ASCII editor. It also adds predefined petcat style PETSCII literals to the default encodings, and enables Unicode letters in symbol names.

For writing sources in UTF-8/UTF-16 encodings this option is required!

```
64tass a.asm

.0000    a9 61          lda #$61        lda #"a"

>0002    31 61 41                       .text "1aA"
>0005    7b 63 6c 65 61 72 7d 74        .text "{clear}text{return}more"
>000e    65 78 74 7b 72 65 74 75
>0016    72 6e 7d 6d 6f 72 65

64tass --ascii a.asm

.0000    a9 41          lda #$41        lda #"a"
>0002    31 41 c1                       .text "1aA"
>0005    93 54 45 58 54 0d 4d 4f        .text "{clear}text{return}more"
>000e    52 45
```

-B, --long-branch

Automatic `BXX *+5 JMP xxx`. Branch too long messages are usually solved by manually rewriting them as `BXX *+5 JMP xxx`. 64tass can do this automatically if this option is used. `BRA` is of course not converted.

```
64tass a.asm
*       = $1000
        bcc $1233       ;error...

64tass a.asm
*       = $1000
        bcs *+5         ;opposite condition
        jmp $1233       ;as simple workaround

64tass --long-branch a.asm
*       = $1000
        bcc $1233       ;no error, automatically converted to the one above
        bcs @b $1233    ;keep this one short regardless and fail if too far
```

-C, **--case-sensitive**
: Make all symbols (variables, opcodes, directives, operators, etc.) case sensitive. Otherwise everything is case insensitive by default.

```
64tass a.asm
label   nop
Label   nop     ;double defined...

64tass --case-sensitive a.asm
label   nop
Label   nop     ;Ok, it's a different label...
```

-D <label>=<value>
: Command line definition.

Same syntax is allowed as in source files. Be careful with strings, the shell might eat the quotes unless escaped.

Using hexadecimal numbers might be tricky as the shell might try to expand them as variables. Either quoting or backslash escaping might help.

In Makefiles all $ signs need to be escaped by doubling them. This needs to be done over the normal shell escaping. For example “`$1000`” becomes “`\$$1000`”.

```
64tass -D ii=2 -D var=\"string\" -D FAST:=true a.asm
        lda #ii ;result: $a9, $02
FAST    :?= false ;define if undefined
```

-q, **--quiet**
: Suppress messages. Disables header and summary messages.

```
64tass --quiet a.asm
```

-T, **--tasm-compatible**
: Enable TASM compatible operators and precedence

Switches the expression evaluator into compatibility mode. This enables “`.`”, “`:`” and “`!`” operators and disables 64tass specific extensions, disables precedence handling and forces 16 bit unsigned evaluation (see “differences to original Turbo Assembler” below)

-I <path>
: Specify include search path

If an included source or binary file can't be found in the directory of the source file then this path is tried. More than one directories can be specified by repeating this option. If multiple matches exist the first one is used.

-M <file>, **--dependencies** <file>

Specify make rule output file

Writes a dependency rules suitable for “make” from the list of files used during compilation.

Please choose source file names which are compatible with Makefiles as there are very little escaping possibilities.

--dependencies-append <file>
: Same as the `--dependencies` option but appends instead of overwrites.

--make-phony
: Enable phony targets for dependencies

This is useful for automatic dependency generation to avoid missing target errors on file rename.

The following Makefile uses the rules generated by 64tass (in “.dep”) to achieve automatic dependency tracking:

```
demo: demo.asm .dep
        64tass --make-phony -M.dep $< -o $@

.dep:
-include .dep
```

7.3 Diagnostic options

Diagnostic message switched start with a “`-W`” and can have an optional “`no-`” prefix to disable them. The options below with this prefix are enabled by default, the others are disabled.

-E <file>, **--error** <file>
: Specify error output file

Normally compilation errors a written to the standard error output. It's possible to redirect them to a file or to the standard output by using “-” as the file name.

--error-append <filename>
: Same as the `--error` option but appends instead of overwrites.

--no-error
: Do not output any error messages, just count them.

-w, **--no-warn**
: Suppress warnings.

Disables warnings during compile. For fine grained diagnostic message suppression see the diagnostic options section.

```
64tass --no-warn a.asm
```

--no-caret-diag
: Suppress displaying of faulty source line and fault position after fault messages.

This is for cases where the fault log is automatically processed and no one ever looks at it and therefore there's no point to display the source lines.

```
64tass --no-caret-diag a.asm
```

--macro-caret-diag
: Restrict source line and fault position display to macro expansions only.

This is for cases where the fault log is processed by an editor which also displays the compilation output somewhere. Only lines which are the result of macro processing will be output to aid debugging. Those which would just duplicate what's in the source editor window will be not.

```
64tass --macro-caret-diag a.asm
```

-Wall
: Enable most diagnostic warnings, except those individually disabled. Or with the “`no-`” prefix disable all except those enabled.

-Werror
: Make all diagnostic warnings to an error, except those individually set to a warning.

-Werror=<name>
: Change a diagnostic warning to an error.

  For example “`-Werror=implied-reg`” makes this check an error. The “`-Wno-error=`” variant is useful with “`-Werror`” to set some to warnings.

-Walias
: Warns about alias opcodes.

  There are several opcodes for the same task, especially for the "6502i" target. This warning helps to find where their use.

-Walign
: Warns when padding bytes were used for alignment.

  Can be used to see where space is wasted for alignment.

-Waltmode
: Warn about alternative address modes.

  Sometimes alternative addressing modes are used as the fitting one is not available. For example there's no lda direct page y so instead data bank y is used with a warning.

-Wbranch-page
: Warns if a branch is crossing a page.

  Page crossing branches execute with a penalty cycle. This option helps to locate them easily.

-Wcase-symbol
: Warn if symbol letter case is used inconsistently.

  This option can be used to enforce letter case matching of symbols in case insensitive mode. This gives similar results to the case sensitive mode (symbols must match exactly) with the main difference of disallowing symbol name definitions differing only in case (these are reported as duplicates).

-Wimmediate
: Warns for cases where immediate addressing is more likely.

  It may be hard to notice if a “`#`” was missed. The code still compiles but there's a huge difference between “`cpx #const`” and “`cpx const`”. Unless the right sort of garbage was on zero page at the time of testing...

  This check might have a lot of false positives if zero page locations are accessed by using small numbers, which is a popular coding style. But there are ways to reduce them.

  For "known" fixed locations `address(x)` can be used, preferably bound to a symbol. Automatic allocation of zero page variables works too (e.g. `zpstuff .byte ?`). And basically everything which is a traditional "label" or derived from a label with an offset.

-Wimplied-reg
: Warns if implied addressing is used instead of register.

  Some instructions have implied aliases like “`asl`” for “`asl a`” for compatibility reasons, but this shorthand is not the preferred form.

-Wleading-zeros
: Warns if about leading zeros.

  A leading zero could be a prefix for an octal number but as octals are not supported the result will be decimal.

-Wlong-branch
: Warns when a long branch is used.

  This option gives a warning for instructions which were modified by the long branch function. Less intrusive than disabling long branches and see where it fails.

-Wmacro-prefix
: Warn about macro call without prefix.

  Such macro calls can easily be mistaken to be labels if invoked without parameters. Also it's hard to notice that an unchanged call turned into label after the definition got renamed. This warning helps to find such calls so that prefixes can be added.

-Wno-deprecated
: Don't warn about deprecated features.

  Unfortunately there were some features added previously which shouldn't have been included. This option disables warnings about their uses.

-Wno-float-compare
: Don't warn if floating point comparisons are only approximate.

  Floating point numbers have a finite precision and comparing them might give unexpected results.

  For example `2.1 + 0.2 == 2.3` is true but gives a warning as the left side is actually bigger by approximately 4.44E−16.

  Normally this is solved by rounding or changing the comparison values.

-Wfloat-round
: Warn when floating point numbers are implicitly rounded.

  A lot of parameters and the data dumping directives need integers but floating point numbers are accepted as well. The style of rounding used may or may not be what you wanted.

  By default floor rounding (to lower) is used and not truncate (towards zero). The reason for this is to enable calculation of fixed point integers by using floating point.

  The difference is subtle and only noticable for negative numbers. The division of `-300/256` is `-2` which matches `floor(-300/256.0)` but not `trunc(-300/256.0)`.

  To get symmetric sine waves around zero `trunc()` needs to be used. Some other calculation might result in `126.9999997` due to inaccuracies in logarithm which would need `round()`.

  To avoid unexpected rounding this option helps to find those places where no explicit rounding was done.

-Wno-ignored
: Don't warn about ignored directives.

-Wno-jmp-bug
: Don't warn about the `jmp ($xxff)` bug.

  With this option it's fine that the high byte is read from the “wrong” address on a 6502, NMOS 6502 and 65DTV02.

```
        jmp (vector)
        .alignpageind vector, 256; jmp bug workaround
vector  .addr ?          ; by avoiding page cross
```

-Wno-label-left
: Don't warn about certain labels not being on left side.

  You may disable this if you use labels which look like mistyped versions of implied addressing mode instructions and you don't want to put them in the first column.

  This check is there to catch typos, unsupported implied instructions, or unknown aliases and not for enforcing label placement.

`-Wno-page`
: Ignore page assertion failures

  Can be used to ignore `.page` assertion block failures. As a middle ground "`-Wno-error=page`" can turn the assertion to a warning only.

`-Wno-pitfalls`
: Don't note about common pitfalls.

  There are some common mistakes, but experts and those who read this don't need extra notes about them. These are:

  Use multi character strings with "`.byte`" instead of "`.text`".
  : This fails because "`.byte`" enforces the 0–255 range for each value.

  Using "`label *=*+1`" style space reservations.
  : Warns as "`*=`" is also the compound multiply operator. The "`*=*+1`" needs to be on a separate line without a label. A better alternatively is to use "`.fill 1`" or "`.byte ?`".

  Negative numbers with "`.byte`" or "`.word`"
  : There are other directives which accept them with proper range checks like "`.char`", "`.sint`".

  Negative numbers with "`lda #xxx`"
  : There's a signed variant for the immediate addressing so "`lda #+xx`" will make it work

`-Wno-portable`
: Don't warn about source portability problems.

  These cross platform development annoyances are checked for:

  - Case insensitive use of file names or use of short names.
  - Use of backslashes for path separation instead of forward slashes.
  - Use of reserved characters in file names.
  - Absolute paths

`-Wno-priority`
: Don't warn about operator priority problems.

  Not all of the unary operators are strongly binding and this may cause surprises. This warning is intended to catch mistakes like this:

```
        .cerror >start != >end; possibly wrong it's >(start != (>end))
        .cerror (>start) != >end; correct high byte check
```

`-Wno-size-larger`
: Don't warn if size is larger due to negative offset

  `size()` and `len()` can be used to measure a memory area. Normally there's no offset used but a positive offset may be used to reduce available space up until nothing remains.

  On the other hand if a negative offset is used then more space will be available (ahead of the area) which may or may not be desired.

```
var     .byte ?, ?, ?
var2    = var - 2       ; start 2 bytes earlier
        ldx #size(var2) ; size is 5 bytes as it's 2 bytes ahead
```

`-Wno-star-assign`
: Don't warn about ignored compound multiply.

  Normally "`symbol *= ...`" means compound multiply of the variable in front. Unfortunately this looks the same a "`label *=*+x`" which is an old-school way to allocate space.

  If the symbol was a variable defined earlier then the multiply is performed without a warning. If it's a new label definition then this warning is used to note that possibly a variable definition was missed earlier.

  If the intention was really a label definition then the "`*=`" can be moved to a separate line, or in case of space allocation it could be improved to use "`.byte ?`" or "`.fill x`".

-Wno-wrap-addr

Don't warn about memory location address space wrap around.

Applying offsets to memory locations may result in addresses which end up outside of the processors address space.

For example `"tmp"` is at `$1000` and then it's addressed as `lda tmp-$2000` then the result will be `lda $f000` or `lda $fff000` depending on the CPU. If this is fine then this warning can be disabled otherwise it can be made into an error by using `-Werror=wrap-addr`.

-Wno-wrap-bank0

Don't warn for bank 0 wrap around.

Adding an offset to a bank 0 address may end up outside of bank 0. If this happens a warning is issued and the address wraps around.

The warning may be ignored using this command line parameter. Alternatively it could be turned into an error by using `-Werror=wrap-bank0`.

-Wno-wrap-dpage

Don't warn for direct page wrap around.

Adding an offset to a direct page address may end up outside of the direct page. For a 65816 or 65EL02 an alternative addressing mode is used but on other processors if this happens a warning is issued and the address wraps around.

The warning may be ignored using this command line parameter. Alternatively it could be turned into an error by using `-Werror=wrap-dpage`.

-Wno-wrap-mem

Don't warn for compile offset wrap around.

While assembling the compile offset may reach the end of memory image. If this happens a warning is issued and the compile offset is set to the start of image.

The warning may be ignored using this command line parameter. Alternatively it could be turned into an error by using `-Werror=wrap-mem`.

The image size depends on the output format. See the Output options section above.

-Wno-wrap-pc

Don't warn for program counter bank crossing.

While assembling the program counter may reach the end of the current program bank. If this happens a warning is issued as a real CPU will not cross the bank on execution. On the other hand some addressing modes handle bank crosses so this might not be actually a problem for data.

The warning may be ignored using this command line parameter. Alternatively it could be turned into an error by using `-Werror=wrap-pc`.

-Wno-wrap-pbank

Don't warn for program bank address calculation wrap around.

Adding an offset to a program bank address may end up outside of the current program bank. If this happens a warning is issued and the address wraps around.

The warning may be ignored using this command line parameter. Alternatively it could be turned into an error by using `-Werror=wrap-pbank`.

-Wold-equal

Warn about old equal operator.

The single "`=`" operator is only there for compatibility reasons and should be written as "`==`" normally.

-Woptimize

Warn about optimizable code.

Warns on things that could be optimized, at least according to the limited analysis done. Currently it's easy to fool with these constructs:

- Self modifying code, especially modifying immediate addressing mode instructions or branch targets
- Using `.byte $2c` and similar tricks to skip instructions.
- Using `*+5` and similar tricks to skip instructions, or to loop like `*-1`.
- Any other method of flow control not involving referenced labels. E.g. calculated returns.
- Register re-mappings on 65DTV02 with SIR and SAC.
- 32 bit operations on 45GS02.

It's also rather simple and conservative, so some opportunities will be missed. Most CPUs are supported with the notable exception of 65816 and 65EL02, but this could improve in later versions.

`-Wshadow`

Warn about symbol shadowing.

Checks if local variables “shadow” other variables of same name in upper scopes in ambiguous ways.

This is useful to detect hard to notice bugs where a new local variable takes the place of a global one by mistake.

```
bl      .block
a       .byte 2         ;'a' is a built-in register
x       .byte 2         ;'x' is a built-in register
        asl a           ; accumulator or the byte above?
        .end
        asl bl.x        ; not ambiguous
```

`-Wstrict-bool`

Warn about implicit boolean conversions.

Boolean values can be interpreted as numeric 0/1 and other types as booleans. This is convenient but may cause mistakes.

To pass this option the following constructs need improvements:

- “`1`” and “`0`” as boolean constants. Use the slightly longer “`true`” and “`false`”.
- Implicit non-zero checks. Write it out like “`.if (lbl & 1) != 0`”.
- Zero checks with “`!`”. Write it out like “`lbl == 0`”.
- Binary operators on booleans. Use the proper “`||`”, “`&&`” and “`^^`” operators.
- Numeric expressions like “`1 + (lbl > 3)`”. It's better as “`(lbl > 3) ? 2 : 1`”.

`-Wunused`

Warn about unused constant symbols.

Symbols which have no references to them are likely redundant. Before removing them check if there's any conditionally compiled out code which might still need them.

The following options can be used to be more specific:

`-Wunused-const`

Warn about unused constants.

`-Wunused-label`

Warn about unused labels.

`-Wunused-macro`

Warn about unused macros.

`-Wunused-variable`

Warn about unused variables.

Symbols which appear in a default 64tass symbol list file and their root symbols are treated as used for exporting purposes.

7.4 Target selection on command line

These options will select the default architecture. It can be overridden by using the ".cpu" directive in the source.

--m65xx
: Standard 65xx (default). For writing compatible code, no extra codes. This is the default.

```
64tass --m65xx a.asm
        lda $14         ;regular instructions
```

-c, --m65c02
: CMOS 65C02. Enables extra opcodes and addressing modes specific to this CPU.

```
64tass --m65c02 a.asm
        stz $d020       ;65c02 instruction
```

--m65ce02
: CSG 65CE02. Enables extra opcodes and addressing modes specific to this CPU.

```
64tass --m65ce02 a.asm
        inz
```

-i, --m6502
: NMOS 65xx. Enables extra illegal opcodes. Useful for demo coding for C64, disk drive code, etc.

```
64tass --m6502 a.asm
        lax $14         ;illegal instruction
```

-t, --m65dtv02
: 65DTV02. Enables extra opcodes specific to DTV.

```
64tass --m65dtv02 a.asm
        sac #$00
```

-x, --m65816
: W65C816. Enables extra opcodes. Useful for SuperCPU projects.

```
64tass --m65816 a.asm
        lda $123456,x
```

-e, --m65el02
: 65EL02. Enables extra opcodes, useful RedPower CPU projects. Probably you'll need "--nostart" as well.

```
64tass --m65el02 a.asm
        lda #0,r
```

--mr65c02
: R65C02. Enables extra opcodes and addressing modes specific to this CPU.

```
64tass --mr65c02 a.asm
        rmb 7,$20
```

--mw65c02
: W65C02. Enables extra opcodes and addressing modes specific to this CPU.

```
64tass --mw65c02 a.asm
        wai
```

--m4510
: CSG 4510. Enables extra opcodes and addressing modes specific to this CPU. Useful for C65 projects.

```
64tass --m4510 a.asm
        map
        eom
```

--m45gs02
45GS02. Enables extra opcodes and addressing modes specific to this CPU. Useful for MEGA65 projects.

```
64tass --m45gs02 a.asm
        ldq $1000
```

7.5 Symbol listing

-l <file>, **--labels**=<file>
List symbols into <file>.

```
64tass -l labels.txt a.asm
*       = $1000
label   jmp label

result (labels.txt):
label           = $1000
```

This option may be used multiple times. In this case the format and root scope options must be placed before this option. Using the same name multiple times is not a good idea.

```
64tass --vice-labels -l all.l --export-labels --labels-root=export -l myexport.inc source.asm
```

This writes symbols for VICE into “all.l” and symbols from scope “export” into “myexport.inc”.

--labels-append=<file>
Same as the `--labels` option but appends instead of overwrites.

--labels-root=<expression>
Specify the scope to list labels from

This option can be used to limit the output to only a subset of labels. The parameter is an expression which must resolve to a namespace. It's usually just the name of a label in the root scope which contains the labels to be listed.

--labels-section=<sectionname>
Specify the section to list labels from

This option can be used to limit the output to a section which code labels refer to.

--labels-add-prefix=<string>
Add prefix to labels

If defined adds a prefix to labels for some formats.

--normal-labels
Lists labels in a 64tass readable format. (default)

List labels without any side effects. Usually for display purposes or for later include.

--export-labels
List labels for include in a 64tass readable format.

The difference to normal symbol listing is that 64tass assumes these symbols will be used in another source. In practice this means that any `.proc`/`.endproc` blocks appearing in the symbol file will always be compiled even if unused otherwise.

--vice-labels
List labels in a VICE readable format.

This format may be used to translate memory locations to something readable in VICE monitor. Therefore simple numeric constants will not show up unless converted to an address first.

VICE symbols may only contain ASCII letters, numbers and underscore. Symbols not meeting this requirement will be omitted.

There's a good chance VICE will complain about already existing labels on import. In the past an attempt was made to filter out such duplicates to eliminate these warnings. However soon it was pointed out that omitted labels are now unavailable for commands like setting breakpoints. As the latter use case is rather more important than some bogus import warnings one has to live with them.

```
64tass --vice-labels -l labels.l a.asm
*       = $1000
label   jmp label

result (labels.l):
al 1000 .label
```

For now colons are used as scope delimiter due to a VICE limitation, but this will be changed to dots in the future.

`--vice-labels-numeric`

List address like symbols in a VICE readable format including numeric constants.

The normal VICE label list does not include symbols like `chrout = $ffd2` or `keybuff = 631` as these are numeric constants and not memory addresses.

Of course there are ways around that. For example:

```
chrout  = address($ffd2)
keybuff = address(631)
*       = $ffd2
chrout  .fill 3
*       = 631
keybuff .fill 10
```

For those who don't want to waste time on explicitly marking addresses as such there's an easy way out by using this command line option.

The tradeoff is that depending on the coding style the label list will become polluted by non-address constants to various degrees. However if one mostly uses numeric constants for addresses only this may be acceptable.

`--dump-labels`

List labels for debugging.

The output will contain symbol locations and paths.

`--simple-labels`

List labels in a simple `label = $x` fashion for interoperatibility.

Somewhat limited but much easier to parse than the normal output with all it's data types.

`--mesen-labels`

List labels in Mesen format

It's a text file in the following format:

<type>:<range>:<name>

- type: the prefix set by the `--labels-add-prefix` command line option
- range: a single hexadecimal number or two with a dash for a multibyte range
- label: name of the label

If the `--labels-section` command line option was given then the range is relative to the section start.

If more than one type of labels need to be listed then the `--labels-append` command line option can be used to append them.

```
64tass --mesen-labels @labeloptions.txt a.asm

Option file (labeloptions.txt):
--labels-section=rom --labels-add-prefix=P --labels labels.mlb
--labels-section=ram --labels-add-prefix=R --labels-append labels.mlb
--labels-section=save --labels-add-prefix=S --labels-append labels.mlb
--labels-section=work --labels-add-prefix=W --labels-append labels.mlb
--labels-section=registers --labels-add-prefix=G --labels-append labels.mlb
```

--ctags-labels

List labels in ctags format

Can be useful to jump to global definitions if the editor supports it.

7.6 Assembly listing

-L <file>, **--list**=<file>

List into <file>. Dumps source code and compiled code into file. Useful for debugging, it's much easier to identify the code in memory within the source files.

```
; 64tass Turbo Assembler Macro V1.5x listing file
; 64tass -L list.txt a.asm
; Fri Dec  9 19:08:55 2005

;Offset ;Hex            ;Monitor        ;Source

;******  Processing input file: a.asm

.1000   a2 00           ldx #$00                ldx #0
.1002   ca              dex             loop    dex
.1003   d0 fd           bne $1002               bne loop
.1005   60              rts                     rts

;******  End of listing
```

--list-append=<file>

Same as the `--list` option but appends instead of overwrites.

-m, **--no-monitor**

Don't put monitor code into listing. There won't be any monitor listing in the list file.

```
; 64tass Turbo Assembler Macro V1.5x listing file
; 64tass --no-monitor -L list.txt a.asm
; Fri Dec  9 19:11:43 2005

;Offset ;Hex            ;Source

;******  Processing input file: a.asm

.1000   a2 00                   ldx #0
.1002   ca              loop    dex
.1003   d0 fd                   bne loop
.1005   60                      rts

;******  End of listing
```

-s, **--no-source**

Don't put source code into listing. There won't be any source listing in the list file.

```
; 64tass Turbo Assembler Macro V1.5x listing file
; 64tass --no-source -L list.txt a.asm
; Fri Dec  9 19:13:25 2005

;Offset ;Hex            ;Monitor
```

```
;******  Processing input file: a.asm

.1000   a2 00           ldx #$00
.1002   ca              dex
.1003   d0 fd           bne $1002
.1005   60              rts

;******  End of listing
```

--line-numbers

This option creates a new column for showing line numbers for easier identification of source origin. The line number is followed with an optional colon separated file number in case it comes from a different file then the previous lines.

```
; 64tass Turbo Assembler Macro V1.5x listing file
; 64tass --line-numbers -L list.txt a.asm
; Fri Dec  9 19:13:25 2005

;Line   ;Offset ;Hex            ;Monitor        ;Source

:1      ;******  Processing input file: a.asm

3       .1000   a2 00           ldx #$00                ldx #0
4       .1002   ca              dex             loop    dex
5       .1003   d0 fd           bne $1002               bne loop
6       .1005   60              rts                     rts

;******  End of listing
```

--tab-size=<number>

By default the listing file is using a tab size of 8 to align the disassembly. This can be changed to other more favorable values like 4. Only spaces are used if 1 is selected. Please note that this has no effect on the source code on the right hand side.

--verbose-list

Normally the assembler tries to minimize listing output by omitting "unimportant" lines. But sometimes it's better to just list everything including comments and empty lines.

```
; 64tass Turbo Assembler Macro V1.5x listing file
; 64tass --verbose-list -L list.txt a.asm
; Fri Dec  9 19:13:25 2005

;Offset ;Hex            ;Monitor        ;Source

;******  Processing input file: a.asm

                                        *       = $1000

.1000   a2 00           ldx #$00                ldx #0
.1002   ca              dex             loop    dex
.1003   d0 fd           bne $1002               bne loop
.1005   60              rts                     rts

;******  End of listing
```

7.7 Other options

-?, **--help**

Give this help list. Prints help about command line options.

--usage

Give a short usage message. Prints short help about command line options.

-V, **--version**

Print program version

7.8 Command line from file

Command line arguments can be read from a file as well. This is useful to store common options for multiple files in one place or to overcome the argument list length limitations of some systems.

The filename needs to be prefixed with an at sign, so “@argsfile” reads options from “argsfile”. It will only work if there's not another file named “@argsfile”. The content is expanded in-place of “@argsfile”.

Stored options must be separated by white space. Single or double quotes can be used in case file names have white space in their names.

Backslash can be used to escape the character following it and it must be used to escape itself. Single and double quotes need to be escaped if needed for string quoting.

Forward slashes can be used as a portable path separation on all systems.

8 Messages

Faults and warnings encountered are sent to the standard error for logging. To redirect them to a file use the “`-E`” command line option. The message format is the following:

<filename>:<line>:<character>: <severity>: <message>

- filename: The name and path of source file where the error happened.
- line: Line number in file, starts from 1.
- character: Character in line, starts from 1. Tabs are not expanded.
- severity: Note, warning, error or fatal.
- message: The fault message itself.

The faulty line will be displayed after the message with a caret pointing to the error location unless this is disabled by using “`--no-caret-diag`” option.

```
a.asm:3:21: error: not defined symbol 'label'
                lda label
                    ^
a.asm:3:21: note: searched in the global scope
```

This is helpful for macro expansions as it displays the processed line which usually looks different to the one in the original source file.

Error buried deep in included files or macros display a backtrace of files after an “In file included from” text where all the files and positions involved are listed down to the main file.

```
In file included from main.asm:3:3:
included.asm:2:11: error: not defined symbol 'test'
                #macro1 test
                        ^
In file included from included.asm:2:3,
                      main.asm:3:3:
macros.asm:3:7: note: original location in an expanded macro was here
                lda test
                    ^
```

Messages ending with “`[-Wxxx]`” are user controllable. This means that using “`-Wno-xxx`” on the command line will silence them and “`-Werror=xxx`” will turn them it into a fault. See Diagnostic options for more details.

8.1 Warnings

`aligned by ? bytes`
alignment was necessary

`approximate floating point`
floating point comparisons are not exact and the numbers were close but maybe not quite

bank 0 address overflow
: the calculated memory location address ended up outside of bank 0 and is now wrapped.

case ignored, value already handled
: this value was already used in an earlier case so here it's ignored

compile offset overflow
: compile continues at the bottom ($0000) as end of compile area was reached

constant result, possibly changeable to 'lda'
: a pre-calculated value could be loaded instead as the result seems to be always the same

could be shorter by using 'xxx' instead
: this shorter instruction gives the same result according to the optimizer

could be simpler by using 'xxx' instead
: this instruction gives the same result but with less dependencies according to the optimizer

deprecated directive, only for TASM compatible mode
: .goto and .lbl should only be used in TASM compatible mode and there are better ways to loop

deprecated equal operator, use '==' instead
: single equal sign for comparisons is going away soon, update source

deprecated modulo operator, use '%' instead
: double slash for modulo is going away soon, update source

deprecated not equal operator, use '!=' instead
: non-standard not equal operators which will stop working in the future, update source

direct page address overflow
: the calculated memory location address ended up outside of direct page and is now wrapped.

directive ignored
: an assembler directive was ignored for compatibility reasons

for ? variables got ? values in ?
: the number of variables must match the number of values when unpacking

file name uses reserved character '?'
: do not use \ : * ? " < > | in file names as some operating systems don't like these

immediate addressing mode suggested
: numeric constant was used as an address which was likely meant as an immediate value

implicit floating point rounding
: a floating point number with fractional part was used for an integer parameter

independent result, possibly changeable to 'lda'
: the result does not seem to depend on the input so it could be just loaded instead

instruction 'xxx' is an alias of 'xxx'
: an alternative instruction name was used

label defined instead of variable multiplication for compatibility
: move the '*=' construct to a separate line or define the variable first as this construct is ambiguous

label not on left side
: check if an instruction name was not mistyped and if the current CPU has it, or remove white space before label

leading zeros ignored
: leading zeros in front of decimals are redundant and don't denote an octal number

long branch used
: branch distance was too long so long branch was used (`bxx *+5 jmp`)

memory location address overflow
: the calculated memory location address ended up outside of the processors address space

not enough values in ?
: all variables in a column oriented for loop should be fed with the same amount of values

over the boundary by ? bytes, aligned by ? bytes
: crossed boundary so alignment was necessary

please separate @b, @w or @l from label or number for future compatibility

future versions will have longer symbols after “@” and so will interpret the immediately following numbers and letters as part if the symbol. Please insert a space between `@b`, `@w` or `@l` and the following label or number now to avoid surprises!

`please use format("%d", ...) as '^' will change it's meaning`
: this operator will be changed to mean the bank byte later, please update your sources

`possible jmp ($xxff) bug with argument ?`
: some 6502 variants read don't increment the high byte on page cross and this may be unexpected

`possibly redundant as ...`
: according to the optimizer this might not be needed

`possibly redundant if last 'jsr' is changed to 'jmp'`
: tail call elimination possibility was detected

`possibly redundant indexing with a constant value`
: the index register used seems to be constant and there's a way to eliminate indexing by a constant offset

`processor program counter crossed bank`
: pc address had crossed into another 64 KiB program bank

`program bank address overflow`
: the calculated memory location address ended up outside of the current program bank and is now wrapped.

`symbol case mismatch '?'`
: the symbol is matching case insensitively but it's not all letters are exactly the same

`the file's real name is not '?'`
: check if all characters match including their case as this is not the real name of the file

`unused symbol '?'`
: this symbol has is not referred anywhere and therefore may be unused

`use '/' as path separation '?'`
: backslash is not a path separator on all systems while forward slash will work independent of the host operating system

`use relative path for '?'`
: file's path is absolute and depends on the file system layout and the source will not compile without the exact same environment

8.2 Errors

`'?' expected`
: something is missing

`? argument is missing`
: not enough arguments supplied

`address in different program bank`
: this instruction is only limited to access the current bank

`address not in processor address space`
: value larger than current CPU address space

`address out of section`
: moving the address around is fine as long as it does not end up before the start of the section

`addressing mode too complex`
: too much indexing or indirection for a valid address

`at least one byte is needed`
: the expression didn't yield any bytes but it's needed here

`block to long for alignment by ? bytes`
: impossible to align if larger than or equals alignment interval

`branch crosses page by ? bytes`
: page crossing was on branch was detected

`branch too far by ? bytes`
: branches have limited range and this went over by some bytes

`can't calculate stable value`
: somehow it's impossible to calculate this expression

`can't calculate this`
: could not get any value, is this a circular reference?

`can't encode character '?' ($xx) in encoding '?'`
: can't translate character in this encoding as no definition was given

`can't get absolute value of`
: not possible to calculate the absolute value of this type

`can't get boolean value of`
: not possible to determine if this value is true or false

`can't get integer value of`
: this value is not a number

`can't get length of`
: this type has no length

`can't get sign of`
: this type does not have a sign as it's not a number

`can't get size of`
: this type has no size

`closing/opening directive '?' not found`
: couldn't find the other half of block directive pair

`conflict`
: at least one feature is provided, which shouldn't be there

`conversion of ? '?' to ? is not possible`
: this type conversion can't be done

`crossing of ? byte page by ? bytes`
: the page directive detected a page cross between start and end directives

`division by zero`
: dividing with zero can't be done

`double defined escape`
: escape sequence already defined in another .edef differently

`double defined range`
: part of a character range was already defined by another .cdef and these ranges can't overlap

`duplicate definition`
: symbol defined more than once

`encoded value ? larger than 8 bit`
: the value for the end of character range for an encoding is too large

`empty list not allowed`
: at least one element is required

`empty range not allowed`
: invalid range but there must be at least one element

`empty string not allowed`
: at least one character is required

`expected exactly/at least/at most ? arguments, got ?`
: wrong number of function arguments used

`expression syntax`
: syntax error

`extra characters on line`
: there's some garbage on the end of line

`floating point overflow`
: infinity reached during a calculation

`format character expected`
: string ended before a format character was found

`general syntax`

can't do anything with this

`index out of range`
: not enough elements in list

`invalid logarithm base`
: logarithm base should be positive and not one

`key not in dictionary`
: key not in the dictionary

`label required`
: a label is mandatory for this directive

`larger than original due to negative offset`
: if a negative offset is used the size gets larger than the original as this effectively adds bytes to the front.

`last byte must not be gap`
: .shift or .shiftl needs a normal byte at the end

`logarithm of non-positive number`
: only positive numbers have a logarithm

`macro call without prefix`
: macro call was found without a prefix and without parameters

`more than a single character`
: no more than a single character is allowed

`more than two characters`
: no more than two characters are allowed

`most significant bit must be clear in byte`
: for .shift and .shiftl only 7 bit "bytes" are valid

`must be used within a loop`
: .break or .continue must be used within a loop

`must be defined later`
: remote alignment must placed before the aligned label

`negative number raised on fractional power`
: can't calculate this

`no ? addressing mode for opcode 'xxx'`
: this addressing mode is not valid for this instruction

`not a bank 0 address`
: value must be a bank zero address

`not a data bank address`
: value must be a data bank address

`not a direct page address`
: value must be a direct page address

`not a key and value pair`
: dictionaries are built from key and value pairs separated by a colon

`not a variable`
: only variables are changeable

`not defined '?'`
: can't find this label at this point

`not hashable`
: the type can't be used as a key in a dictionary

`not in range -1.0 to 1.0`
: the function is only valid in the -1.0 to 1.0 range

`not iterable`
: value is not a list or other iterable object

`not measurable as start offset beyond size of original`
: the applied offset was larger than the original size. For example if size(data) is 2 then size(data + 1) is 1. However size(data + 3) makes no sense as there's no such thing as a nega-

tive size.

`offset out of range`
: code offset too much

`operands could not be broadcast together with shapes ? and ?`
: list length must match or must have a single element only

`ptext too long by ? bytes`
: .ptext is limited to 255 bytes maximum

`requirements not met`
: not all features are provided, at least one is missing

`reserved symbol name '?'`
: do not use this symbol name

`shadow definition`
: symbol is defined in an upper scope as well and is used ambiguously

`some operation '?' of type '?' and type '?' not possible`
: can't do this calculation with these values

`square root of negative number`
: can't calculate the square root of a negative number

`start ? not on same ? byte page as end ?`
: the endpage directive detected a mismatch of page to the page directive

`too large for a ? bit signed/unsigned integer`
: value out of range

`unexpected character '?'`
: unexpected control or Unicode character

`unknown processor '?'`
: unknown CPU name

`unknown argument name '?'`
: no parameter argument known like this

`unknown format character '?'`
: no format character known like this

`use '?' instead of '?'`
: wrong sort of character was used. For example a left double quotation mark was used instead of a regular quotation mark.

`value needs to be non-negative`
: only positive numbers or zero is accepted here

`wrong type <?>`
: wrong object type used

`zero raised to negative power`
: can't calculate this

`zero value not allowed`
: do not use zero for example with .null

8.3 Fatal errors

`can't open file`
: cannot open file

`can't write ? file '?'`
: cannot write an output file

`compilation was interrupted`
: shows the line where the interruption happened

`error reading file`
: error while reading

`file recursion`
: wrong nesting of .include

`function recursion too deep`

wrong use of nested functions

`macro recursion too deep`
: wrong use of nested macros

`option '?' doesn't allow an argument`
: command line option doesn't need any argument

`option '?' is ambiguous`
: command line option abbreviation is too short

`option '?' not recognized`
: no such command line option

`option '?' requires an argument`
: command line option needs an argument

`out of memory`
: won't happen ;)

`section '?' for output not found`
: the section given on command line couldn't be found

`too many passes`
: with a carefully crafted source file it's possible to create unresolvable situations but try to avoid this

`unknown option '?'`
: option not known

`weak recursion`
: excessive nesting of .weak

9 Credits

Original 6502tass written for DOS by Marek Matula of Taboo.

It was ported to ANSI C by BigFoot/Breeze. This is when it's name changed to 64tass.

Soci/Singular reworked the code over the years to the point that practically nothing was left from original at this point.

Improved TASS compatibility, PETSCII codes by Groepaz.

Additional code: my_getopt command-line argument parser by Benjamin Sittler, avl tree code by Franck Bui-Huu, ternary tree code by Daniel Berlin, snprintf Alain Magloire, Amiga OS4 support files by Janne Peräaho.

Pierre Zero helped to uncover a lot of faults by fuzzing. Also there were a lot of discussions with oziphantom about the need of various features.

Main developer and maintainer: soci at c64.rulez.org

10 Default translation and escape sequences

10.1 Raw 8-bit source

By default raw 8-bit encoding is used and nothing is translated or escaped. This mode is for compiling sources which are already PETSCII.

10.1.1 The “none” encoding for raw 8-bit

Does no translation at all, no translation table, no escape sequences.

10.1.2 The “screen” encoding for raw 8-bit

The following translation table applies, no escape sequences.

Input	Byte	Input	Byte
`00-1F`	`80-9F`	`20-3F`	`20-3F`

Table 43: Built-in PETSCII to PETSCII screen code translation table

Input	Byte	Input	Byte
40-5F	00-1F	60-7F	40-5F
80-9F	80-9F	A0-BF	60-7F
C0-FE	40-7E	FF	5E

10.2 Unicode and ASCII source

Unicode encoding is used when the “-a” option is given on the command line.

10.2.1 The “none” encoding for Unicode

This is a Unicode to PETSCII mapping, including escape sequences for control codes.

Glyph	Unicode	Byte	Glyph	Unicode	Byte
-@	U+0020-U+0040	20-40	A-Z	U+0041-U+005A	C1-DA
[	U+005B	5B	]	U+005D	5D
a-z	U+0061-U+007A	41-5A	£	U+00A3	5C
π	U+03C0	FF	←	U+2190	5F
↑	U+2191	5E	─	U+2500	C0
│	U+2502	DD	┌	U+250C	B0
┐	U+2510	AE	└	U+2514	AD
┘	U+2518	BD	├	U+251C	AB
┤	U+2524	B3	┬	U+252C	B2
┴	U+2534	B1	┼	U+253C	DB
╭	U+256D	D5	╮	U+256E	C9
╯	U+256F	CB	╰	U+2570	CA
╱	U+2571	CE	╲	U+2572	CD
╳	U+2573	D6	▁	U+2581	A4
▂	U+2582	AF	▃	U+2583	B9
▄	U+2584	A2	▌	U+258C	A1
▍	U+258D	B5	▎	U+258E	B4
▏	U+258F	A5	▒	U+2592	A6
▔	U+2594	A3	▕	U+2595	A7
▖	U+2596	BB	▗	U+2597	AC
▘	U+2598	BE	▚	U+259A	BF
▝	U+259D	BC	○	U+25CB	D7
●	U+25CF	D1	◤	U+25E4	A9
◥	U+25E5	DF	♠	U+2660	C1
♣	U+2663	D8	♥	U+2665	D3
♦	U+2666	DA	✓	U+2713	BA
🭰	U+1FB70	D4	🭱	U+1FB71	C7
🭲	U+1FB72	C2	🭳	U+1FB73	DD
🭴	U+1FB74	C8	🭵	U+1FB75	D9
🭶	U+1FB76	C5	🭷	U+1FB77	C4
🭸	U+1FB78	C3	🭹	U+1FB79	C0
🭺	U+1FB7A	C6	🭻	U+1FB7B	D2
🭼	U+1FB7C	CC	🭽	U+1FB7D	CF
🭾	U+1FB7E	D0	🭿	U+1FB7F	BA
🮂	U+1FB82	B7	🮃	U+1FB83	B8
🮇	U+1FB87	AA	🮈	U+1FB88	B6
🮌	U+1FB8C	DC	🮏	U+1FB8F	A8
🮕	U+1FB95	FF	🮘	U+1FB98	DF
🮙	U+1FB99	A9			

Table 44: Built-in Unicode to PETSCII translation table

Escape	Byte	Escape	Byte	Escape	Byte
{bell}	07	{black}	90	{blk}	90
{blue}	1F	{blu}	1F	{brn}	95
{brown}	95	{cbm-*}	DF	{cbm-+}	A6
{cbm--}	DC	{cbm-0}	30	{cbm-1}	81
{cbm-2}	95	{cbm-3}	96	{cbm-4}	97

Table 45: Built-in PETSCII escape sequences

Escape	Byte	Escape	Byte	Escape	Byte
{cbm-5}	98	{cbm-6}	99	{cbm-7}	9A
{cbm-8}	9B	{cbm-9}	29	{cbm-@}	A4
{cbm-^}	DE	{cbm-a}	B0	{cbm-b}	BF
{cbm-c}	BC	{cbm-d}	AC	{cbm-e}	B1
{cbm-f}	BB	{cbm-g}	A5	{cbm-h}	B4
{cbm-i}	A2	{cbm-j}	B5	{cbm-k}	A1
{cbm-l}	B6	{cbm-m}	A7	{cbm-n}	AA
{cbm-o}	B9	{cbm-pound}	A8	{cbm-p}	AF
{cbm-q}	AB	{cbm-r}	B2	{cbm-s}	AE
{cbm-t}	A3	{cbm-up arrow}	DE	{cbm-u}	B8
{cbm-v}	BE	{cbm-w}	B3	{cbm-x}	BD
{cbm-y}	B7	{cbm-z}	AD	{clear}	93
{clr}	93	{control-0}	92	{control-1}	90
{control-2}	05	{control-3}	1C	{control-4}	9F
{control-5}	9C	{control-6}	1E	{control-7}	1F
{control-8}	9E	{control-9}	12	{control-:}	1B
{control-;}	1D	{control-=}	1F	{control-@}	00
{control-a}	01	{control-b}	02	{control-c}	03
{control-d}	04	{control-e}	05	{control-f}	06
{control-g}	07	{control-h}	08	{control-i}	09
{control-j}	0A	{control-k}	0B	{control-left arrow}	06
{control-l}	0C	{control-m}	0D	{control-n}	0E
{control-o}	0F	{control-pound}	1C	{control-p}	10
{control-q}	11	{control-r}	12	{control-s}	13
{control-t}	14	{control-up arrow}	1E	{control-u}	15
{control-v}	16	{control-w}	17	{control-x}	18
{control-y}	19	{control-z}	1A	{cr}	0D
{cyan}	9F	{cyn}	9F	{delete}	14
{del}	14	{dish}	08	{down}	11
{ensh}	09	{esc}	1B	{f10}	82
{f11}	84	{f12}	8F	{f1}	85
{f2}	89	{f3}	86	{f4}	8A
{f5}	87	{f6}	8B	{f7}	88
{f8}	8C	{f9}	80	{gray1}	97
{gray2}	98	{gray3}	9B	{green}	1E
{grey1}	97	{grey2}	98	{grey3}	9B
{grn}	1E	{gry1}	97	{gry2}	98
{gry3}	9B	{help}	84	{home}	13
{insert}	94	{inst}	94	{lblu}	9A
{left arrow}	5F	{left}	9D	{lf}	0A
{lgrn}	99	{lower case}	0E	{lred}	96
{lt blue}	9A	{lt green}	99	{lt red}	96
{orange}	81	{orng}	81	{pi}	FF
{pound}	5C	{purple}	9C	{pur}	9C
{red}	1C	{return}	0D	{reverse off}	92
{reverse on}	12	{rght}	1D	{right}	1D
{run}	83	{rvof}	92	{rvon}	12
{rvs off}	92	{rvs on}	12	{shift return}	8D
{shift-*}	C0	{shift-+}	DB	{shift-,}	3C
{shift--}	DD	{shift-.}	3E	{shift-/}	3F
{shift-0}	30	{shift-1}	21	{shift-2}	22
{shift-3}	23	{shift-4}	24	{shift-5}	25
{shift-6}	26	{shift-7}	27	{shift-8}	28
{shift-9}	29	{shift-:}	5B	{shift-;}	5D
{shift-@}	BA	{shift-^}	DE	{shift-a}	C1
{shift-b}	C2	{shift-c}	C3	{shift-d}	C4
{shift-e}	C5	{shift-f}	C6	{shift-g}	C7
{shift-h}	C8	{shift-i}	C9	{shift-j}	CA
{shift-k}	CB	{shift-l}	CC	{shift-m}	CD
{shift-n}	CE	{shift-o}	CF	{shift-pound}	A9

Escape	Byte	Escape	Byte	Escape	Byte
{shift-p}	D0	{shift-q}	D1	{shift-r}	D2
{shift-space}	A0	{shift-s}	D3	{shift-t}	D4
{shift-up arrow}	DE	{shift-u}	D5	{shift-v}	D6
{shift-w}	D7	{shift-x}	D8	{shift-y}	D9
{shift-z}	DA	{space}	20	{sret}	8D
{stop}	03	{swlc}	0E	{swuc}	8E
{tab}	09	{up arrow}	5E	{up/lo lock off}	09
{up/lo lock on}	08	{upper case}	8E	{up}	91
{white}	05	{wht}	05	{yellow}	9E
{yel}	9E				

10.2.2 The “screen” encoding for Unicode

This is a Unicode to PETSCII screen code mapping, including escape sequences for control code screen codes.

Glyph	Unicode	Translated	Glyph	Unicode	Translated
-?	U+0020-U+003F	20-3F	@	U+0040	00
A-Z	U+0041-U+005A	41-5A	[	U+005B	1B
]	U+005D	1D	a-z	U+0061-U+007A	01-1A
£	U+00A3	1C	π	U+03C0	5E
←	U+2190	1F	↑	U+2191	1E
─	U+2500	40	│	U+2502	5D
┌	U+250C	70	┐	U+2510	6E
└	U+2514	6D	┘	U+2518	7D
├	U+251C	6B	┤	U+2524	73
┬	U+252C	72	┴	U+2534	71
┼	U+253C	5B	╭	U+256D	55
╮	U+256E	49	╯	U+256F	4B
╰	U+2570	4A	╱	U+2571	4E
╲	U+2572	4D	╳	U+2573	56
▁	U+2581	64	▂	U+2582	6F
▃	U+2583	79	▄	U+2584	62
▌	U+258C	61	▍	U+258D	75
▎	U+258E	74	▏	U+258F	65
▒	U+2592	66	▔	U+2594	63
▕	U+2595	67	▖	U+2596	7B
▗	U+2597	6C	▘	U+2598	7E
▚	U+259A	7F	▝	U+259D	7C
○	U+25CB	57	●	U+25CF	51
◤	U+25E4	69	◥	U+25E5	5F
♠	U+2660	41	♣	U+2663	58
♥	U+2665	53	♦	U+2666	5A
✓	U+2713	7A	🭰	U+1FB70	54
🭱	U+1FB71	47	🭲	U+1FB72	42
🭳	U+1FB73	5D	🭴	U+1FB74	48
🭵	U+1FB75	59	🭶	U+1FB76	45
🭷	U+1FB77	44	🭸	U+1FB78	43
🭹	U+1FB79	40	🭺	U+1FB7A	46
🭻	U+1FB7B	52	🭼	U+1FB7C	4C
🭽	U+1FB7D	4F	🭾	U+1FB7E	50
🭿	U+1FB7F	7A	🮂	U+1FB82	77
🮃	U+1FB83	78	🮇	U+1FB87	6A
🮈	U+1FB88	76	🮌	U+1FB8C	5C
🮏	U+1FB8F	68	🮕	U+1FB95	5E
🮘	U+1FB98	5F	🮙	U+1FB99	69

Table 46: Built-in Unicode to PETSCII screen code translation table

Escape	Byte	Escape	Byte	Escape	Byte
{cbm-*}	5F	{cbm-+}	66	{cbm--}	5C

Table 47: Built-in PETSCII screen code escape sequences

Escape	Byte	Escape	Byte	Escape	Byte
{cbm-0}	30	{cbm-9}	29	{cbm-@}	64
{cbm-^}	5E	{cbm-a}	70	{cbm-b}	7F
{cbm-c}	7C	{cbm-d}	6C	{cbm-e}	71
{cbm-f}	7B	{cbm-g}	65	{cbm-h}	74
{cbm-i}	62	{cbm-j}	75	{cbm-k}	61
{cbm-l}	76	{cbm-m}	67	{cbm-n}	6A
{cbm-o}	79	{cbm-pound}	68	{cbm-p}	6F
{cbm-q}	6B	{cbm-r}	72	{cbm-s}	6E
{cbm-t}	63	{cbm-up arrow}	5E	{cbm-u}	78
{cbm-v}	7E	{cbm-w}	73	{cbm-x}	7D
{cbm-y}	77	{cbm-z}	6D	{left arrow}	1F
{pi}	5E	{pound}	1C	{shift-*}	40
{shift-+}	5B	{shift-,}	3C	{shift--}	5D
{shift-.}	3E	{shift-/}	3F	{shift-0}	30
{shift-1}	21	{shift-2}	22	{shift-3}	23
{shift-4}	24	{shift-5}	25	{shift-6}	26
{shift-7}	27	{shift-8}	28	{shift-9}	29
{shift-:}	1B	{shift-;}	1D	{shift-@}	7A
{shift-^}	5E	{shift-a}	41	{shift-b}	42
{shift-c}	43	{shift-d}	44	{shift-e}	45
{shift-f}	46	{shift-g}	47	{shift-h}	48
{shift-i}	49	{shift-j}	4A	{shift-k}	4B
{shift-l}	4C	{shift-m}	4D	{shift-n}	4E
{shift-o}	4F	{shift-pound}	69	{shift-p}	50
{shift-q}	51	{shift-r}	52	{shift-space}	60
{shift-s}	53	{shift-t}	54	{shift-up arrow}	5E
{shift-u}	55	{shift-v}	56	{shift-w}	57
{shift-x}	58	{shift-y}	59	{shift-z}	5A
{space}	20	{up arrow}	1E		

11 Opcodes

11.1 Standard 6502 opcodes

ADC	61 65 69 6D 71 75 79 7D	**AND**	21 25 29 2D 31 35 39 3D
ASL	06 0A 0E 16 1E	**BCC**	90
BCS	B0	**BEQ**	F0
BIT	24 2C	**BMI**	30
BNE	D0	**BPL**	10
BRK	00	**BVC**	50
BVS	70	**CLC**	18
CLD	D8	**CLI**	58
CLV	B8	**CMP**	C1 C5 C9 CD D1 D5 D9 DD
CPX	E0 E4 EC	**CPY**	C0 C4 CC
DEC	C6 CE D6 DE	**DEX**	CA
DEY	88	**EOR**	41 45 49 4D 51 55 59 5D
INC	E6 EE F6 FE	**INX**	E8
INY	C8	**JMP**	4C 6C
JSR	20	**LDA**	A1 A5 A9 AD B1 B5 B9 BD
LDX	A2 A6 AE B6 BE	**LDY**	A0 A4 AC B4 BC
LSR	46 4A 4E 56 5E	**NOP**	EA
ORA	01 05 09 0D 11 15 19 1D	**PHA**	48
PHP	08	**PLA**	68
PLP	28	**ROL**	26 2A 2E 36 3E
ROR	66 6A 6E 76 7E	**RTI**	40
RTS	60	**SBC**	E1 E5 E9 ED F1 F5 F9 FD
SEC	38	**SED**	F8
SEI	78	**STA**	81 85 8D 91 95 99 9D
STX	86 8E 96	**STY**	84 8C 94

Table 48: The standard 6502 opcodes

TAX	AA	**TAY**	A8
TSX	BA	**TXA**	8A
TXS	9A	**TYA**	98

ASL	0A	**BGE**	B0
BLT	90	**CMP**	C0 C4 CC E0 E4 EC
CPA	C1 C5 C9 CD D1 D5 D9 DD	**GCC**	4C 90
GCS	4C B0	**GEQ**	4C F0
GGE	4C B0	**GLT**	4C 90
GMI	30 4C	**GNE**	4C D0
GPL	10 4C	**GVC**	4C 50
GVS	4C 70	**LDR**	8A 98 A0 A1 A2 A4 A5 A6 A8 A9 AA AC AD AE B1 B4 B5 B6 B9 BA BC BD BE
LSR	4A	**ORR**	01 05 09 0D 11 15 19 1D
PSH	08 48	**PUL**	28 68
ROL	2A	**ROR**	6A
SHL	06 0A 0E 16 1E	**SHR**	46 4A 4E 56 5E
STR	81 84 85 86 8C 8D 8E 91 94 95 96 99 9A 9D		

Table 49: 6502 aliases, pseudo instructions

11.2 6502 illegal opcodes

This processor is a standard 6502 with the NMOS illegal opcodes.

ANC	0B	**ANE**	8B
ARR	6B	**ASR**	4B
DCP	C3 C7 CF D3 D7 DB DF	**ISB**	E3 E7 EF F3 F7 FB FF
JAM	02	**LAX**	A3 A7 AB AF B3 B7 BF
LDS	BB	**NOP**	04 0C 14 1C 80
RLA	23 27 2F 33 37 3B 3F	**RRA**	63 67 6F 73 77 7B 7F
SAX	83 87 8F 97	**SBX**	CB
SHA	93 9F	**SHS**	9B
SHX	9E	**SHY**	9C
SLO	03 07 0F 13 17 1B 1F	**SRE**	43 47 4F 53 57 5B 5F

Table 50: Additional NMOS 6502 opcodes

AHX	93 9F	**ALR**	4B
AXS	CB	**DCM**	C3 C7 CF D3 D7 DB DF
INS	E3 E7 EF F3 F7 FB FF	**ISC**	E3 E7 EF F3 F7 FB FF
LAE	BB	**LAS**	BB
LXA	AB	**TAS**	9B
XAA	8B		

Table 51: Additional NMOS 6502 aliases

11.3 65DTV02 opcodes

This processor is an enhanced version of standard 6502 with some illegal opcodes.

BRA	12	**SAC**	32
SIR	42		

Table 52: Additional 65DTV02 opcodes

GRA	12 4C		

Table 53: Additional 65DTV02 pseudo instructions

ANC	0B	**JAM**	02
LDS	BB	**NOP**	04 0C 14 1C 80
SBX	CB	**SHA**	93 9F
SHS	9B	**SHX**	9E
SHY	9C		

Table 54: These illegal opcodes are not valid

AHX	93 9F	**AXS**	CB

Table 55: These aliases are not valid

LAE	BB	LAS	BB
TAS	9B		

11.4 Standard 65C02 opcodes

This processor is an enhanced version of standard 6502.

ADC	72	AND	32
BIT	34 3C 89	BRA	80
CMP	D2	DEC	3A
EOR	52	INC	1A
JMP	7C	LDA	B2
ORA	12	PHX	DA
PHY	5A	PLX	FA
PLY	7A	SBC	F2
STA	92	STZ	64 74 9C 9E
TRB	14 1C	TSB	04 0C

Table 56: Additional 65C02 opcodes

CLR	64 74 9C 9E	CPA	D2
DEA	3A	GRA	4C 80
INA	1A	LDR	B2
ORR	12	PSH	5A DA
PUL	7A FA	STR	64 74 92 9C 9E

Table 57: Additional 65C02 aliases and pseudo instructions

11.5 R65C02 opcodes

This processor is an enhanced version of standard 65C02.

Please note that the bit number is not part of the instruction name (like `RMB7 $20`). Instead it's the first element of coma separated parameters (e.g. `RMB 7,$20`).

BBR	0F 1F 2F 3F 4F 5F 6F 7F	BBS	8F 9F AF BF CF DF EF FF
NOP	44 54 82 DC	RMB	07 17 27 37 47 57 67 77
SMB	87 97 A7 B7 C7 D7 E7 F7		

Table 58: Additional R65C02 opcodes over 65C02

11.6 W65C02 opcodes

This processor is an enhanced version of R65C02.

STP	DB	WAI	CB

Table 59: Additional W65C02 opcodes over R65C02

HLT	DB		

Table 60: Additional W65C02 aliases over R65C02

11.7 W65816 opcodes

This processor is an enhanced version of 65C02.

ADC	63 67 6F 73 77 7F	AND	23 27 2F 33 37 3F
BRL	82	CMP	C3 C7 CF D3 D7 DF
COP	02	EOR	43 47 4F 53 57 5F
JMP	5C DC	JSL	22
JSR	FC	LDA	A3 A7 AF B3 B7 BF
MVN	54	MVP	44
ORA	03 07 0F 13 17 1F	PEA	F4
PEI	D4	PER	62
PHB	8B	PHD	0B
PHK	4B	PLB	AB

Table 61: Additional W65816 opcodes over 65C02

PLD	2B	**REP**	C2
RTL	6B	**SBC**	E3 E7 EF F3 F7 FF
SEP	E2	**STA**	83 87 8F 93 97 9F
STP	DB	**TCD**	5B
TCS	1B	**TDC**	7B
TSC	3B	**TXY**	9B
TYX	BB	**WAI**	CB
WDM	42	**XBA**	EB
XCE	FB		

CLP	C2	**CPA**	C3 C7 CF D3 D7 DF
CSP	02	**HLT**	DB
JML	5C DC	**LDR**	3B 7B 9B A3 A7 AF B3 B7 BB BF
ORR	03 07 0F 13 17 1F	**PSH**	0B 4B 8B D4 F4
PUL	2B AB	**STR**	1B 83 87 8F 93 97 9F
SWA	EB	**TAD**	5B
TAS	1B	**TDA**	7B
TSA	3B		

Table 62: Additional W65816 aliases over 65C02

11.8 65EL02 opcodes

This processor is an enhanced version of standard 65C02.

ADC	63 67 73 77	**AND**	23 27 33 37
CMP	C3 C7 D3 D7	**DIV**	4F 5F 6F 7F
ENT	22	**EOR**	43 47 53 57
JSR	FC	**LDA**	A3 A7 B3 B7
MMU	EF	**MUL**	0F 1F 2F 3F
NXA	42	**NXT**	02
ORA	03 07 13 17	**PEA**	F4
PEI	D4	**PER**	62
PHD	DF	**PLD**	CF
REA	44	**REI**	54
REP	C2	**RER**	82
RHA	4B	**RHI**	0B
RHX	1B	**RHY**	5B
RLA	6B	**RLI**	2B
RLX	3B	**RLY**	7B
SBC	E3 E7 F3 F7	**SEA**	9F
SEP	E2	**STA**	83 87 93 97
STP	DB	**SWA**	EB
TAD	BF	**TDA**	AF
TIX	DC	**TRX**	AB
TXI	5C	**TXR**	8B
TXY	9B	**TYX**	BB
WAI	CB	**XBA**	EB
XCE	FB	**ZEA**	8F

Table 63: Additional 65EL02 opcodes over 65C02

CLP	C2	**CPA**	C3 C7 D3 D7
HLT	DB	**LDR**	9B A3 A7 AB AF B3 B7 BB DC
ORR	03 07 13 17	**PSH**	D4 DF F4
PUL	CF	**RSH**	0B 1B 44 4B 54 5B
RUL	2B 3B 6B 7B	**STR**	83 87 93 97

Table 64: Additional 65EL02 aliases over 65C02

11.9 65CE02 opcodes

There's a small deviation in the use of `TAD` and `TDA` for the base page. That's consistent with the direct page addressing mode, `.dpage` directive and convention of other processors. Therefore `TAB` and `TBA` mnemonics are aliases only.

For similar reason the base page register needs to be referenced as `D` instead of `B` as the latter is the data bank register on other processors.

As later variants have both left and right rotating multi byte instructions the rotate left word `ROW` mnemonic should be written as `RLW` in new code to avoid a potential confusion.

ASR	43 44 54	**ASW**	CB
BCC	93	**BCS**	B3
BEQ	F3	**BMI**	33
BNE	D3	**BPL**	13
BRA	83	**BSR**	63
BVC	53	**BVS**	73
CLE	02	**CPZ**	C2 D4 DC
DEW	C3	**DEZ**	3B
INW	E3	**INZ**	1B
JSR	22 23	**LDA**	E2
LDZ	A3 AB BB	**NEG**	42
PHW	F4 FC	**PHZ**	DB
PLZ	FB	**RLW**	EB
RTS	62	**SEE**	03
STA	82	**STX**	9B
STY	8B	**TAD**	5B
TAZ	4B	**TDA**	7B
TSY	0B	**TYS**	2B
TZA	6B		

Table 65: Additional 65CE02 opcodes over R65C02

ASR	43	**BGE**	B3
BLT	93	**CMP**	C2 D4 DC
LDR	0B 4B 6B 7B A3 AB BB E2	**NEG**	42
ROW	EB	**RTN**	62
STR	2B 5B 82 8B 9B	**TAB**	5B
TBA	7B		

Table 66: Additional 65CE02 aliases over R65C02

CLR	64 74 9C 9E		

Table 67: Aliases of R65C02 not valid for 65CE02

11.10 CSG 4510 opcodes

This processor is an enhanced version of 65CE02.

MAP	5C		

Table 68: Additional CSG 4510 opcodes over 65CE02

EOM	EA		

Table 69: Additional CSG 4510 aliases over 65CE02

11.11 45GS02 opcodes

Unfortunately the original naming convention couldn't be adopted as it used four letter mnemonics for most new operations. If these longer names are preferred it's quite easy to define macros named like that using `.segment` and “`\@`”.

The four letter mnemonics got shortened so that all register `Q` operations are ending with the name of the register. That leaves two letters to describe the function which isn't much.

Alternatively the generic instruction syntax can be used with register `Q` parameter to get more descriptive and familiar mnemonics.

ADQ	**ADC Q,**	Add with carry to register `Q`	(also known as `ADCQ`)
ANQ	**AND Q,**	Logical and register `Q`	(also known as `ANDQ`)
BTQ	**BIT Q,**	Bit test register `Q`	(also known as `BITQ`)
CPQ	**CMP Q,**	Compare register `Q`	(also known as `CMPQ`)

Table 70: Q register memory 45GS02 mnemonics

EOQ	**EOR Q,**	Exclusive or register `Q`	(also known as `EORQ`)
LDQ	**LDR Q,**	Load register `Q`	
ORQ	**ORR Q,**	Logical or register `Q`	
SBQ	**SBC Q,**	Subtract with carry from register `Q`	(also known as `SBCQ`)
STQ	**STR Q,**	Store register `Q`	

The `Q` register modify instructions detach the register from the mnemonic just like `ASL A`. The four letter convention had it in both the mnmemonic and the parameter which is redundant.

	ASL Q	Arithmetic shift left register `Q`	(also known as `ASRQ Q`)
	ASR Q	Arithmetic shift right register `Q`	(also known as `ASLQ Q`)
DEQ	**DEC Q**	Decrement register `Q`	(also known as `DEQ Q`)
INQ	**INC Q**	Increment register `Q`	(also known as `INQ Q`)
	LSR Q	Logical shift right register `Q`	(also known as `LSRQ Q`)
	ROL Q	Rotate left register `Q`	(also known as `ROLQ Q`)
	ROR Q	Rotate right register `Q`	(also known as `RORQ Q`)

Table 71: Q register modify 45GS02 mnemonics

The `INQ` and `DEQ` mnemonics have only implied addressing mode just like `INX` or `INY`. These can't be used on memory as register `Q` is not involved in the operation. No `Q` register parameter either, the name makes it clear already.

The double word memory modify instructions are ending with the letter `D`. That's consistent with pre-existing word operations like `INW`, `DEW`, `ASW` and others.

ARD		Arithmetic shift right double word	(also known as `ASRQ`)
ASD		Arithmetic shift left double word	(also known as `ASLQ`)
DED		Decrement double word	(also known as `DEQ`)
IND		Increment double word	(also known as `INQ`)
LSD		Logical shift right double word	(also known as `LSRQ`)
RLD		Rotate left double word	(also known as `ROLQ`)
RRD		Rotate right double word	(also known as `RORQ`)

Table 72: Double word memory modify 45GS02 mnemonics

ADC	72	**ADQ**	65 6D 72
AND	32	**ANQ**	25 2D 32
ARD	43 44 54	**ASD**	0A 06 0E 16 1E
ASL	0A	**ASR**	43
BTQ	24 2C	**CMP**	D2
CPQ	C5 CD D2	**DEC**	3A
DED	3A C6 CE D6 DE	**DEQ**	3A
EOQ	45 4D 52	**EOR**	52
INC	1A	**IND**	1A E6 EE F6 FE
INQ	1A	**LDA**	B2
LDQ	A5 AD B2	**LSD**	46 4A 4E 56 5E
LSR	4A	**ORA**	12
ORQ	05 0D 12	**RLD**	26 2A 2E 36 3E
ROL	2A	**ROR**	6A
RRD	66 6A 6E 76 7E	**SBC**	F2
SBQ	E5 ED F2	**STA**	92
STQ	85 8D 92		

Table 73: Additional 45GS02 opcodes over CSG 4510

CPA	D2	**LDR**	A5 AD B2
ORR	05 0D 12	**SHL**	0A
SHR	4A	**STR**	85 8D 92

Table 74: Additional 45GS02 aliases over CSG 4510

12 Appendix

12.1 Assembler directives

.addr .al .align .alignblk .alignind .alignpageind .as .assert .autsiz .bend .binary .binclude .bfor .block .break .breakif .brept .bwhile .byte .case .cdef .cerror .char .check .comment .continue .continueif .cpu .cwarn .databank .default .dint .dpage .dsection .dstruct .dunion .dword .edef .elif .else .elsif .enc .encode .end .endblock .endc .endalignblk .endcomment .endencode .endf .endfor .endfunction .endif .endlogical .endm .endmacro .endn .endnamespace .endp .endpage .endproc .endrept .ends .endsection .endsegment .endstruct .endswitch .endu .endunion .endv .endvirtual .endweak .endwhile .endwith .eor .error .fi .fill .for .from .function .goto .here .hidemac .if .ifeq .ifmi .ifne .ifpl .include .lbl .lint .logical .long .macro .mansiz .namespace .next .null .offs .option .page .pend .proc .proff .pron .ptext .rept .rta .section .seed .segment .send .sfunction .shift .shiftl .showmac .sint .struct .switch .tdef .text .union .var .virtual .warn .weak .while .with .word .xl .xs

12.2 Built-in functions

abs acos addr all any asin atan atan2 binary byte cbrt ceil char cos cosh deg dint dword exp floor format frac hypot len lint log log10 long pow rad random range repr round rta sign sin sinh sint size sort sqrt tan tanh trunc word

12.3 Built-in types

address bits bool bytes code dict float gap int list str symbol tuple type